

AD-AuthGraph: Authorization State Analysis in Active Directory

Contents

Page number

• Introduction to AD-AuthGraph.....	4
• The Blues.local Domain Environment.....	4
• State 0 – Baseline Authorization State.....	5
◦ Initial Group Memberships	
◦ Kerberoastable Service Account Configuration	
◦ Collecting Raw Active Directory Data	
◦ Executing the AD-AuthGraph Pipeline	
◦ HTML Report Analysis - State 0	
• State 1 – Tier-0 Exposure and ACL Introduction.....	13
◦ Director Group Nested in Domain Admins	
◦ Function vs Security Protocol	
◦ Understanding ACLs and Their Security Impact	
◦ ACL Permissions Applied in State 1	
◦ ACL Verification Using PowerShell	
◦ Collecting State 1 Authorization Data	
◦ Executing the AD-AuthGraph Pipeline	
◦ HTML Report Analysis - State 1	
• State 2 – ResetPassword Privilege Escalation.....	25
◦ ResetPassword Extended Right Explained	
◦ Granting ResetPassword to mdsiraj	
◦ Collecting State 2 Authorization Data	
◦ Executing the AD-AuthGraph Pipeline	
◦ HTML Report Analysis - State 2	
◦ Privilege Escalation Path Analysis	
• State 3 – Escalation Chain Expansion Through WriteDACL.....	32
◦ WriteDACL on mdsiraj	

- Authorization Chain Extension
 - Verification and Data Collection
 - Executing the AD-AuthGraph Pipeline
 - HTML Report Analysis - State 3
 - Escalation Path Analysis
- State 4 – ResetPassword Chaining Through Kerberoastable Identity.....39
 - Granting ResetPassword to rjadeja on rsharma
 - Verification and Data Collection
 - Executing the AD-AuthGraph Pipeline
 - HTML Report Analysis - State 4
 - Extended Privilege Escalation Analysis
- State 5 – WriteProperty Abuse on Tier-0 Nested Group.....44
 - Granting WriteProperty on Director Group
 - Verification and Data Collection
 - Executing the AD-AuthGraph Pipeline
 - HTML Report Analysis - State 5
 - Privilege Escalation Path Analysis
- State Comparison and Security Evolution.....50
 - Comparison Engine Overview
 - Using the --compare Feature
 - Comparison Output Files
 - State Comparisons
 - Security Evolution Across Authorization States

AD-AuthGraph

A pipeline of PowerShell and Python scripts which map authorization in an Active Directory environment. Active Directory, developed by Microsoft, is the most widely used directory service which manages users, computers, and security permissions within a network, from a central database. Within an Active Directory environment, authorization is a critical function after authentication (verifying identity). Authorization determines what kind of access and permissions users have over the resources (files, applications, servers etc.) in a network. This not only impacts hierarchy but also security in an Active Directory domain. AD AuthGraph maps authorization in a domain, primarily ACLs (Access Control Lists) and their security implications.

The Domain – Blues.local

AD-AuthGraph was tested in the local Active Directory (AD) lab which consisted of one Windows Server 2025 Domain Controller, two Windows 11 hosts(all three virtual machines). Blues.local domain was created with the built-in groups eight custom user accounts and seven custom groups were created to resemble a small single domain AD environment.

Custom users:

1. vkohli
2. jbumrah
3. rsharma
4. rashwin
5. arahane
6. rjadeja
7. rpant
8. mdsiraj

Custom groups:

1. Administration (not be confused with the built in Administrators)
2. Director
3. Finance

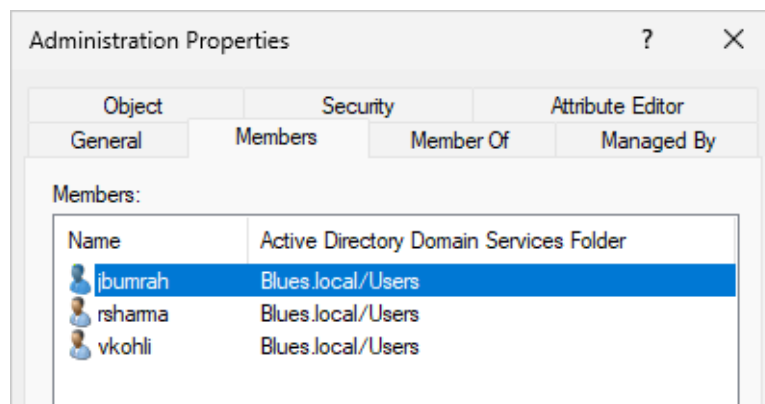
4. HR
5. IT
6. Sales
7. Helpdesk

State_0 - The baseline

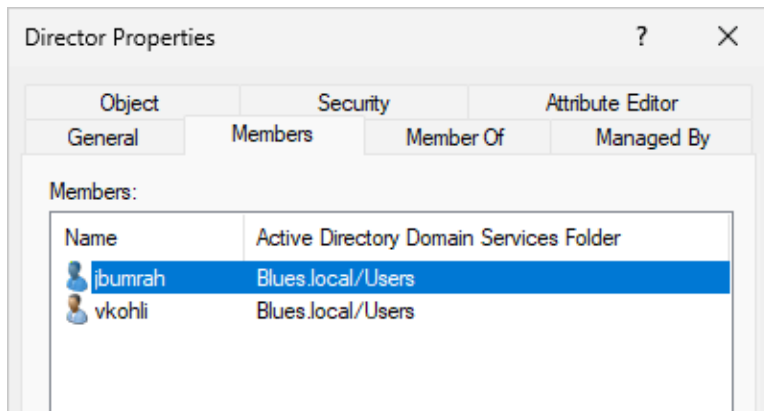
At this stage no custom ACL abuse permissions or privileged group nesting have been applied..

The users have been assigned group memberships within the custom groups.

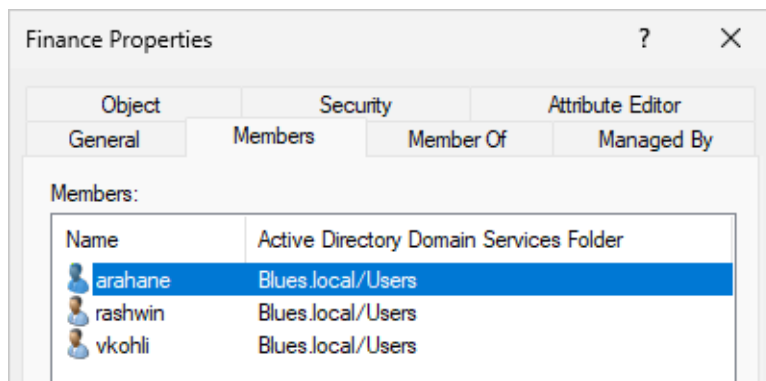
1. Administration - vkohli, jbumrah, rsharma
2. Director - vkohli, jbumrah
3. Finance - vkohli, rashwin, arahane
4. HR - rsharma
5. IT - rashwin, arahane, rjadeja
6. Helpdesk - mdsiraj
7. Sales - vkohli, jbumrah, rpant



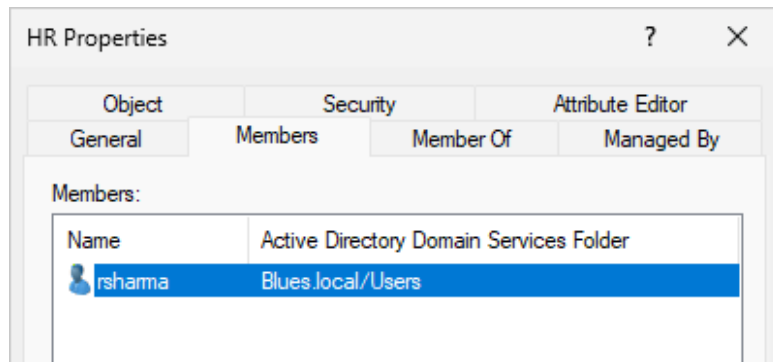
Administration group members



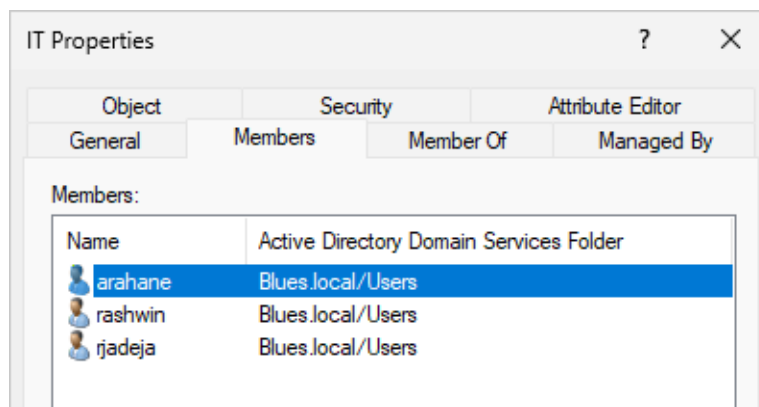
Director group members



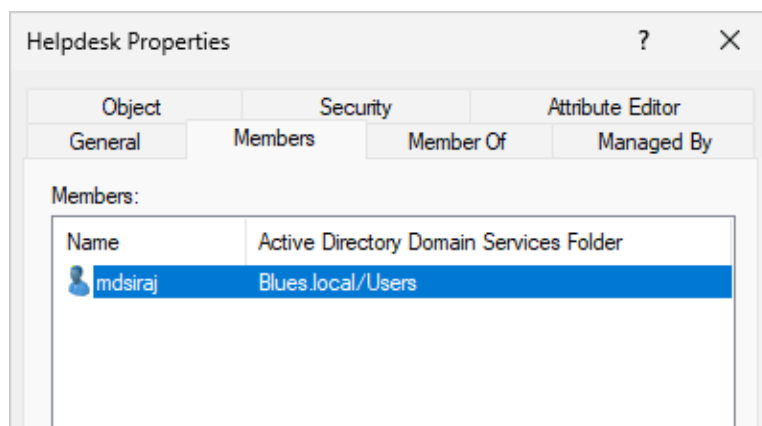
Finance group members



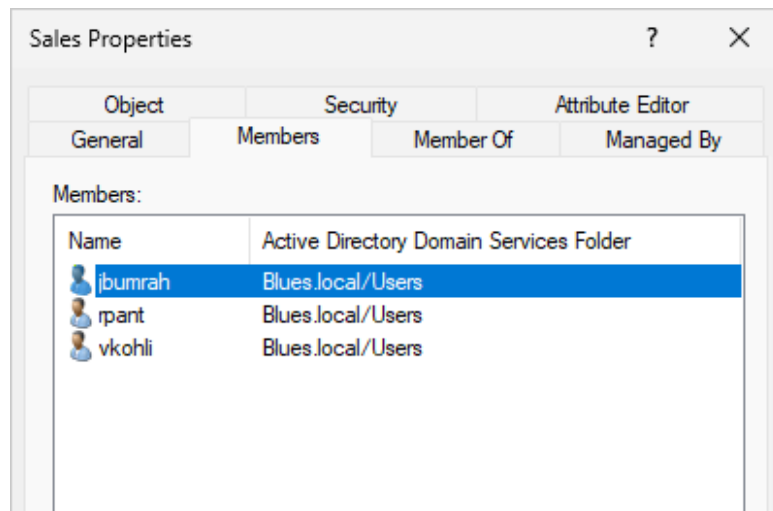
HR group members



IT group members



Helpdesk group members



Sales group members

In an Active Directory network some user accounts are assigned service identity. These accounts can be susceptible to Kerberoasting attacks.

Here, the user rsharma is assigned Service Principal Name (SPN) *HTTP/hr-app.blues.local* which associates it with the HTTP service in Active Directory.

```
setspn -s HTTP/hr-app.blues.local blues\rsharma
```

```
PS C:\WINDOWS\system32> setspn -L blues\rsharma
Registered ServicePrincipalNames for CN=rsharma,CN=Users,DC=Blues,DC=local:
HTTP/hr-app.blues.local
PS C:\WINDOWS\system32>
```

SPN assigned successfully

AD-AuthGraph executed at State 0

Collect the AD raw data with Powershell collector script

```
.\Collect-ADAuthorizationState.ps1 -Domain "Blues.local" -Outputfile "state_0.json"
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> .\Collect-ADAuthorizationState.ps1 -Domain "Blues.local" -Outputfile "state_0.json"
[+] ActiveDirectory module loaded

=== Active Directory Authorization State Collection ===
Domain: Blues.local
Output: state_0.json

[*] Collecting users...
[+] Collected 11 users
[*] Collecting groups...
[+] Collected 58 groups
[*] Collecting computers...
[+] Collected 3 computers
[*] Collecting group memberships (including nested)...
[+] Collected 46 group memberships
[*] Collecting Service Principal Names (SPNs)...
[+] Collected 15 SPNs
[*] Collecting delegation configurations...
[+] Collected 1 delegation configurations
[*] Collecting ACL permissions (this may take a while)...
[*] Processed ACLs for 10 users...
[*] Processed ACLs for 10 groups...
[*] Processed ACLs for 20 groups...
[*] Processed ACLs for 30 groups...
[*] Processed ACLs for 40 groups...
[*] Processed ACLs for 50 groups...
[+] Collected 458 ACL permissions
```

Terminal response for Collect-ADAuthorizationState.ps1

```
[*] Exporting to JSON...
[+] Export successful: state_0.json (449.98 KB)

=== Collection Summary ===
Users:                11
Groups:               58
Computers:            3
Group Memberships:    46
SPNs:                 15
Delegations:          1
ACL Permissions:      458

[+] Collection complete!
Next step: python main.py --state state_0.json

PS C:\Users\Administrator\AD_Authorization_Graph_project>
```

Terminal response for Collect-ADAuthorizationState.ps1

The raw AD data in generated file state_0.json

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python main.py
-a----- 5/4/2026 5:07 PM 460784 state_0.json
```

state_0.json generated

Executing the Python pipeline graph_builder.py → analyzer.py → refiner.py → visualizer.py with main.py

```
python main.py state_0.json --output .\reports\
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project\reports> cd ..
PS C:\Users\Administrator\AD_Authorization_Graph_project> python main.py state_0.json --output .\reports\
=====
AD AUTHORIZATION GRAPH ANALYSIS PIPELINE
=====
Input: state_0.json
Output: reports

[STEP 1/4] Building Authorization Graph
=====
[*] Loading authorization state from: state_0.json
    Domain: Blues.local
    Collection Date: 2026-05-04 17:07:22
[+] Graph built successfully
    Nodes: 88
    Edges: 252
[+] Graph data exported to: reports\state_0_graph.json
[+] Graph exported: reports\state_0_graph.json
[+] Graph Statistics:
    Nodes: 88
    Edges: 252
    Users: 11
    Groups: 58
    Computers: 3

[STEP 2/4] Analyzing Authorization Graph
=====
[*] Loading graph from: reports\state_0_graph.json
[+] Loaded 88 nodes
[+] Loaded 252 edges
```

Terminal response to execution of main.py

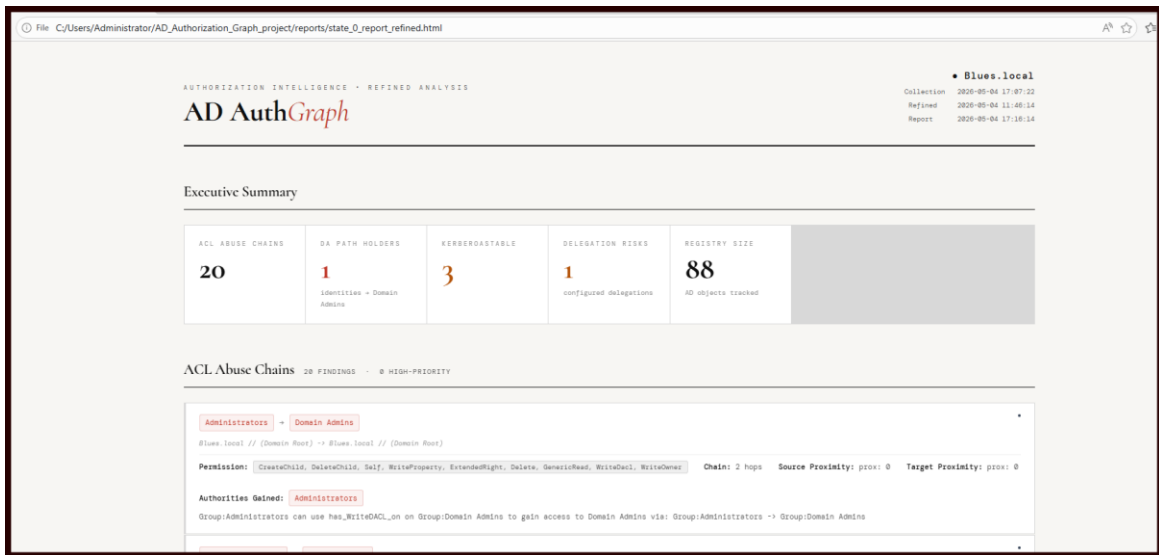
```
[STEP 2/4] Analyzing Authorization Graph
-----
[*] Loading graph from: reports\state_0_graph.json
  [+] Loaded 88 nodes
  [+] Loaded 252 edges
  Domain: Blues.local
  Collection Date: 2026-05-04 17:07:22
  [+] Graph loaded: 88 nodes, 252 edges
  [+] Report saved to: reports\state_0_analysis.json

  [+] Analysis Summary:
    Domain Admin Paths: 1
    Kerberoastable Identities: 3
    ACL Abuse Chains: 7
    Delegation Risks: 1
  [+] Analysis saved: reports\state_0_analysis.json

[STEP 3/4] Refining Analysis (OU ACL Path Context)
-----
[*] Loading analysis: reports\state_0_analysis.json
[*] Building IdentityRegistry ...
  [+] IdentityRegistry: 88 nodes loaded from graph JSON
  [+] Tier-0 anchors identified: 12
  [+] Using REVERSE edges: 15 nodes reach Tier 0
[*] Running refinement passes ...
  [+] DA path map: 1 identities mapped
  [+] Kerberoastable OU map: 3 entries
  [+] Delegation OU map: 1 entries
  [+] OU risk heatmap: 2 distinct OU paths
```

Terminal response to execution of main.py

Screenshots of the generated HTML report (state_0_report_refined.html)



Refined HTML report generated - for State 0

Domain Admin Privilege Paths

1 IDENTITIES

IDENTITY	SHORTEST PATH	PATHS	HOPS	PROX	OU	BFS
Administrator	Administrator → Domain Admins	17	2	1	Blues.local // (Domain Root)	✓

Kerberoastable Identities

3 IDENTITIES

0 HIGH-PRIORITY

krbtgt

MEDIUM

DISABLED

prox: -

admin/changepw

OU: Blues.local // (Domain Root)

SPN Count: 1

DA Paths: 0

isshime

MEDIUM

ENABLED

prox: -

HTTP/hr-app.blues.local

OU: Blues.local // (Domain Root)

SPN Count: 1

DA Paths: 0

BLUES-DC\$

MEDIUM

ENABLED

prox: -

Dfsr-12f9a27c-bf97-4787-9364-d3180c55e804/Blues-DC.Blues.local

ldap/Blues-DC.Blues.local/ForestDnsZones.Blues.local

ldap/Blues-DC.Blues.local/DomainDnsZones.Blues.local

DNS/Blues-DC.Blues.local

GC/Blues-DC.Blues.local/Blues.local

RPC/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b_msdcs.Blues.local

E3514235-4806-11D1-AB04-00C04FC2DCD2/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b/Blues.local

ldap/BLUES-DC/BLUES

ldap/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b_msdcs.Blues.local

ldap/Blues-DC.Blues.local/BLUES

ldap/BLUES-DC

ldap/Blues-DC.Blues.local

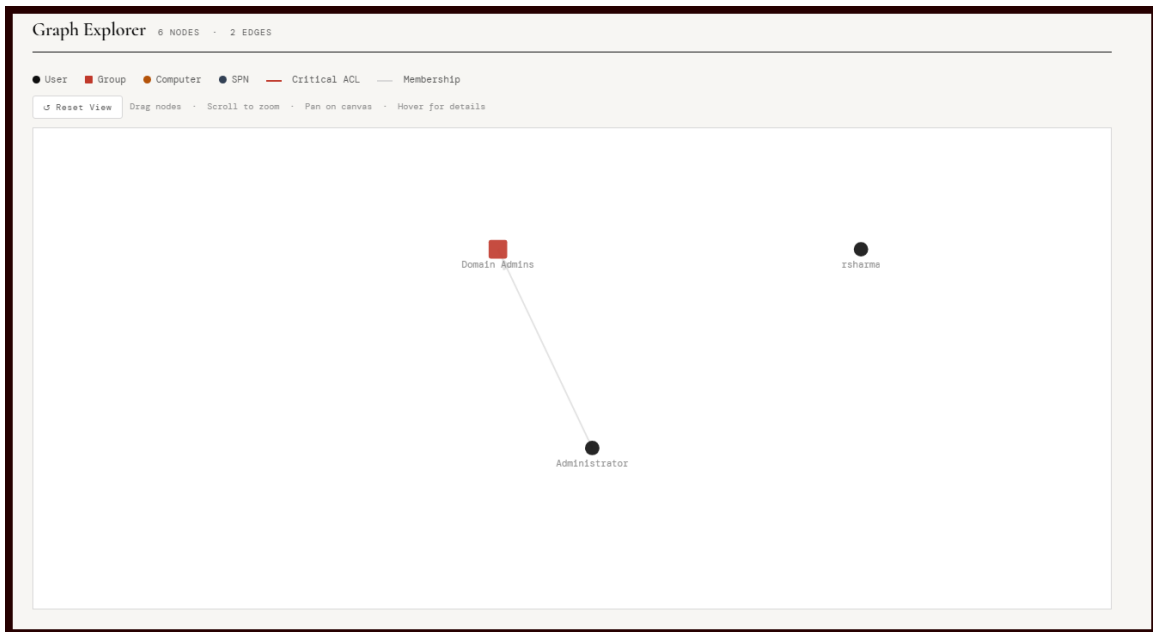
ldap/Blues-DC.Blues.local/Blues.local

OU: Blues.local // Domain Controllers

SPN Count: 13

DA Paths: 0

Domain Privilege Paths and Kerberoastable identities section in HTML report



Graph Explorer in HTML report displays paths to Domain Admins

We can observe from AD-AuthGraph's state_0 output that none of the custom users have Domain Admin privileges nor escalation paths. Only the DC hosts Administrator has Domain Admin right.

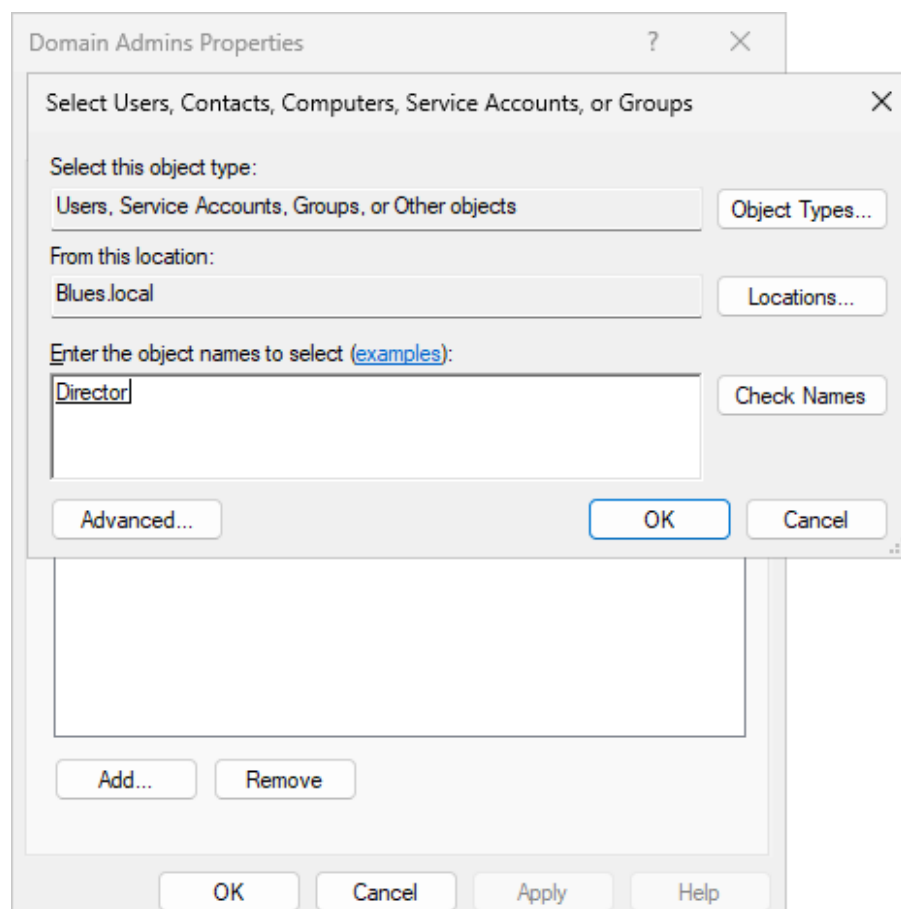
State 1

Custom group Director is nested in built-in Domain Admins.

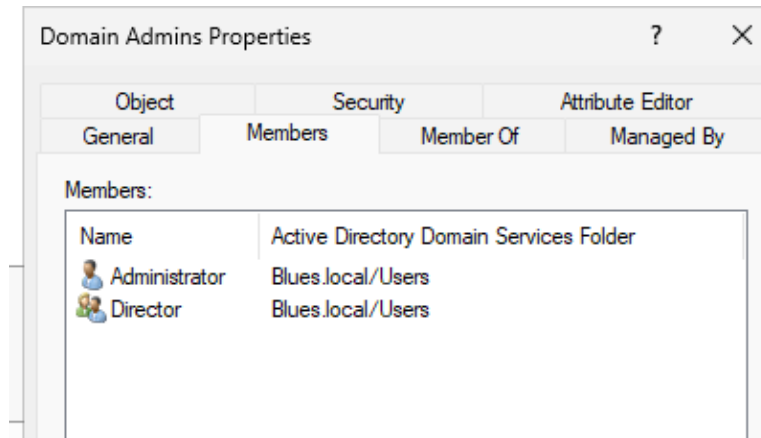
This elevates users vkohli and jbumrah to Domain Admin privilege.

In Active Directory when a group is made member_of another group, then the members of the first group inherit the permissions and privileges of the containing group. This is also known as Group nesting.

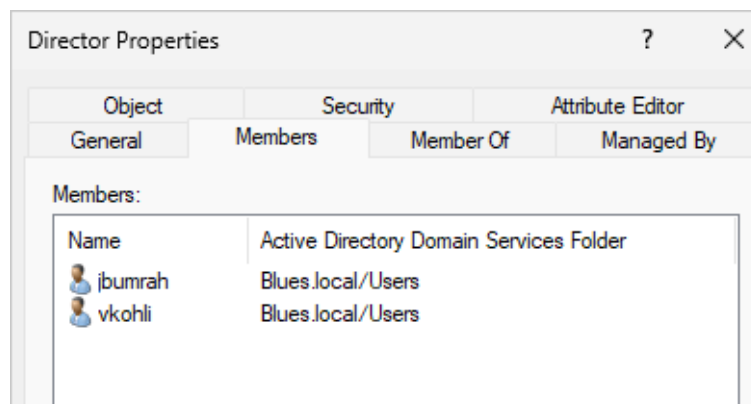
Here, we can see that the members vkohli and jbumrah of the custom group Director will inherit Domain Admin privilege as the Director group is made a member_of (nested) in group Domain Admins.



Director being made member of Domain Admins group



Director nested in Domain Admins group



Members of Director group

Before applying the ACLs Function vs Security protocol was defined. The domain must be functional and secure. Not extreme in either OR category. - Define functional requirements (what must work)

- Helpdesk and IT can reset passwords
- IT must manage workstations
- Administration must manage servers
- Directors must have Tier-0 authority These are non-negotiable capabilities.
- Define security invariants (what must never happen)
 - No non-Director will be given Domain Admins membership

- Business groups (Finance/HR/Sales) cannot influence Tier-0
- No user outside IT can modify IT OU
- No kerberoastable account can hold Tier-0 authority

Understanding ACLs and their impact:

- What are ACLs?

ACLs (Access Control Lists) are lists containing ACE (Access Control Entries) which define permissions given to objects (users, groups and computers etc.) in an Active Directory network. These permissions determine what are their authority over other objects in the domain.

- Types of ACLs applied in State 1:

- **WriteDacl** - Write Discretionary Access Control List.

- What is a DACL?

If access to an object has have restrictions then the object requires a DACL. , otherwise in case of a NULL DACL (No DACL) the system allows full access on the object to all. ACE (Access Control Entries) inside the DACL determines which users or groups have access rights on the object. If the object has DACL but no ACE (Empty DACL) then the system denies access to all. When there is an ACE; those listed have access while all other trustees are denied.

WriteDacl is the permission to modify the DACL, the permitted user can potentially add, remove or modify the ACE and allow himself GenericAll (full access) over the object.

- **GenericAll** - in simplified terms means full and complete access and control. The user who has GenericAll on an object. It is a powerfull permission granted through an ACE on the target object. It permits the trustee to manage, modify or even delete the target object.
- **WriteProperty** - is a specific access right which allows the permitted user to modify certain attributes(properties) of the target object. It is not full control. But for example a user with WriteProperty on the attribute to add

members on a high-value group can add himself or any other to group, instantly elevating privileges.

- **GenericWrite** - is a broad write permission which allows modification on many writable attributes of a target object. It can be more dangerous than WriteProperty from security point-of-view. This is because it applies to multiple attributes of the target object. Although, it is not equivalent to full control; still enables privilege escalation often.
- **ReadProperty** - allows the permitted user to read the attributes (properties) of the target object. Although, not as powerful as the already discussed ACLs, it can play an important role in enumeration of the target object.

Using a short Powershell script the following ACLs were applied:

```
vkohli:
ReadProperty on Administration
WriteDacl on Director
GenericAll on Finance
WriteDacl on Sales
```

To verify if the ACLs were applied correctly, used the following Powershell code:

```
(Get-Acl "AD:CN=Finance,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "vkohli"}
# Replace Finance with other group names, and "vkohli" with other user
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Finance,CN=Users,DC=blues,DC=local").Access |
Where-Object {$_.IdentityReference -match "vkohli"}

ActiveDirectoryRights : GenericAll
```

vkohli has GenericAll on Finance group

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Sales,CN=Users,DC=blues,DC=local").Access | Wh
ere-Object {$_.IdentityReference -match "vkohli"}

ActiveDirectoryRights : WriteDacl
```

vkohli has WriteDacl on Sales group

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Administration,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "vkohli"}
```

```
ActiveDirectoryRights : ReadProperty
InheritanceType       : None
```

vkohli has ReadProperty on Administration group

```
jbumrah:
ReadProperty on Administration
WriteDacl on Director
WriteDacl on Sales
```

Verification:

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Administration,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "jbumrah"}
```

```
ActiveDirectoryRights : ReadProperty
InheritanceType       : None
```

jbumrah has ReadProperty on Administration group

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Sales,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "jbumrah"}
```

```
ActiveDirectoryRights : WriteDacl
```

jbumrah has WriteDacl on Sales group

```
rsharma:
ReadProperty on Administration
GenericAll on HR
```

Verification:

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=HR,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rsharma"}
```

```
ActiveDirectoryRights : GenericAll
```

rsharma has GenericAll on HR group

```

ess | Where-Object {$_.IdentityReference -match "rsharma"}
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Administration,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rsharma"}

```

```
ActiveDirectoryRights : ReadProperty
```

rsharma has ReadProperty on Administration group

```

rashwin:
WriteProperty Finance
GenericWrite on IT

```

Verification:

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Finance,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rashwin"}

```

```
ActiveDirectoryRights : WriteProperty
```

rashwin has WriteProperty on Finance group

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=IT,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rashwin"}

```

rashwin has GenericWrite on IT

```

arahane:
ReadProperty on Finance
WriteProperty on IT

```

Verification:

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Finance,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "arahane"}
```

```
ActiveDirectoryRights : WriteProperty
```

arahane has WriteProperty on Finance

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=IT,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "arahane"}
```

```
ActiveDirectoryRights : WriteProperty
```

arahane has WriteProperty on IT

```
rjadeja:  
WriteProperty on IT
```

Verification:

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=IT,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rjadeja"}
```

```
ActiveDirectoryRights : WriteProperty
```

rjadeja has WriteProperty on IT

Verification:

```
rpant:  
WriteProperty on Sales
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Sales,CN=Users,DC=blues,DC=local").Access | Where-Object {$_.IdentityReference -match "rpant"}
```

```
ActiveDirectoryRights : WriteProperty
```

rpant has WriteProperty on Sales

```
mdsiraj:
WriteProperty on Helpdesk
```

Verification:

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> (Get-Acl "AD:CN=Helpdesk,CN=Users,DC=blues,DC=local").Access |
Where-Object {$_.IdentityReference -match "mdsiraj"}

ActiveDirectoryRights : WriteProperty
```

mdsiraj has WriteProperty on Helpdesk

Run the Powershell collector - Collect-ADAuthorizationState.ps1 to collect the raw AD data

```
.\Collect-ADAuthorization.ps1 "state_1.json"
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> .\Collect-ADAuthorizationState.ps1 "state_1.json"
[+] ActiveDirectory module loaded

=== Active Directory Authorization State Collection ===
Domain: BLUES.LOCAL
Output: state_1.json

[*] Collecting users...
    [+] Collected 11 users
[*] Collecting groups...
    [+] Collected 58 groups
[*] Collecting computers...
    [+] Collected 3 computers
[*] Collecting group memberships (including nested)...
    [+] Collected 55 group memberships
[*] Collecting Service Principal Names (SPNs)...
    [+] Collected 15 SPNs
[*] Collecting delegation configurations...
    [+] Collected 1 delegation configurations
[*] Collecting ACL permissions (this may take a while)...
    [*] Processed ACLs for 10 users...
    [*] Processed ACLs for 10 groups...
    [*] Processed ACLs for 20 groups...
    [*] Processed ACLs for 30 groups...
    [*] Processed ACLs for 40 groups...
    [*] Processed ACLs for 50 groups...
```

Collect-ADAuthorizationState.ps1 terminal response

Execute the Python pipeline with main.py

```
python main.py .\state_1.json --output .\reports
# the output files including the HTML report will be stored in a deicated folder
```

```
[+] Collection complete!
Next step: python main.py --state state_1.json

PS C:\Users\Administrator\AD_Authorization_Graph_project> python main.py .\state_1.json --output .\reports\
=====
AD AUTHORIZATION GRAPH ANALYSIS PIPELINE
=====
Input: state_1.json
Output: reports

[STEP 1/4] Building Authorization Graph
=====
[*] Loading authorization state from: state_1.json
Domain: BLUES.LOCAL
Collection Date: 2026-05-04 18:35:19
[+] Graph built successfully
Nodes: 88
Edges: 261
[+] Graph data exported to: reports\state_1_graph.json
[+] Graph exported: reports\state_1_graph.json
[+] Graph Statistics:
Nodes:      88
Edges:     261
Users:      11
Groups:     58
Computers:  3

[STEP 2/4] Analyzing Authorization Graph
=====
```

Python pipeline execution with main.py terminal response for graph_builder.py

```
[STEP 2/4] Analyzing Authorization Graph
=====
[*] Loading graph from: reports\state_1_graph.json
[+] Loaded 88 nodes
[+] Loaded 261 edges
Domain: BLUES.LOCAL
Collection Date: 2026-05-04 18:35:19
[+] Graph loaded: 88 nodes, 261 edges
[+] Report saved to: reports\state_1_analysis.json

[+] Analysis Summary:
Domain Admin Paths:      3
Kerberoastable Identities:3
ACL Abuse Chains:       10
Delegation Risks:       1
[+] Analysis saved: reports\state_1_analysis.json

[STEP 3/4] Refining Analysis (OU ACL Path Context)
=====
[*] Loading analysis: reports\state_1_analysis.json
[*] Building IdentityRegistry ...
[+] IdentityRegistry: 88 nodes loaded from graph JSON
[+] Tier-0 anchors identified: 12
[+] Using REVERSE edges: 18 nodes reach Tier 0
[*] Running refinement passes ...
[+] DA path map: 3 identities mapped
[+] Kerberoastable OU map: 3 entries
[+] Delegation OU map: 1 entries
[+] OU risk heatmap: 2 distinct OU paths
```

Python pipeline execution with main.py terminal response for analyzer.py and refiner.py

```
Administrator: Windows Powe X + v
[+] Refined output saved: reports\state_1_refined.json

[+] Refinement Summary:
Registry size:      88 nodes
ACL findings (total): 32
High-priority ACL:  0
Identities with DA paths: 3
Kerberoastable:    3
Delegation risks:  1
[+] Refined JSON saved: reports\state_1_refined.json

[STEP 4/4] Generating Interactive HTML Report
-----
[+] HTML report saved: reports\state_1_report_refined.html
    Size: 77.6 KB

[OPTIONAL] Graph PNG (SKIPPED - matplotlib not available)
           pip install matplotlib to enable

=====
PIPELINE COMPLETE
=====
Time Elapsed: 1.45 seconds

Generated Files:
1. Graph JSON:    reports\state_1_graph.json
2. Analysis JSON: reports\state_1_analysis.json
3. Refined JSON:  reports\state_1_refined.json
4. HTML Report:   reports\state_1_report_refined.html
```

Python pipeline execution with main.py terminal response for visualizer.py.

File C:\Users\Administrator\AD_Authorization_Graph_project\reports\state_1_report_refined.html

AUTHORIZATION INTELLIGENCE • REFINED ANALYSIS

AD AuthGraph

• BLUES.LOCAL

Collection 2020-05-04 18:35:19
Refined 2020-05-04 13:11:22
Report 2020-05-04 18:41:22

Executive Summary

ACL ABUSE CHAINS	DA PATH HOLDERS	KERBEROASTABLE	DELEGATION RISKS	REGISTRY SIZE
32	3 Identities → Domain Admins	3	1 configured delegations	88 AD objects tracked

ACL Abuse Chains 32 FINDINGS • 0 HIGH-PRIORITY

Administrators → Domain Admins

BLUES.LOCAL // (Domain Root) → BLUES.LOCAL // (Domain Root)

Permission: CreateChild, DeleteChild, Self, WriteProperty, ExtendedRight, Delete, GenericRead, WriteDACL, WriteOwner

Chain: 2 hops Source Proximity: prox: 0 Target Proximity: prox: 0

Authorities Gained: Administrators

Group:Administrators can use hasWriteDACL on Group:Domain Admins to gain access to Domain Admins via: Group:Administrators → Group:Domain Admins

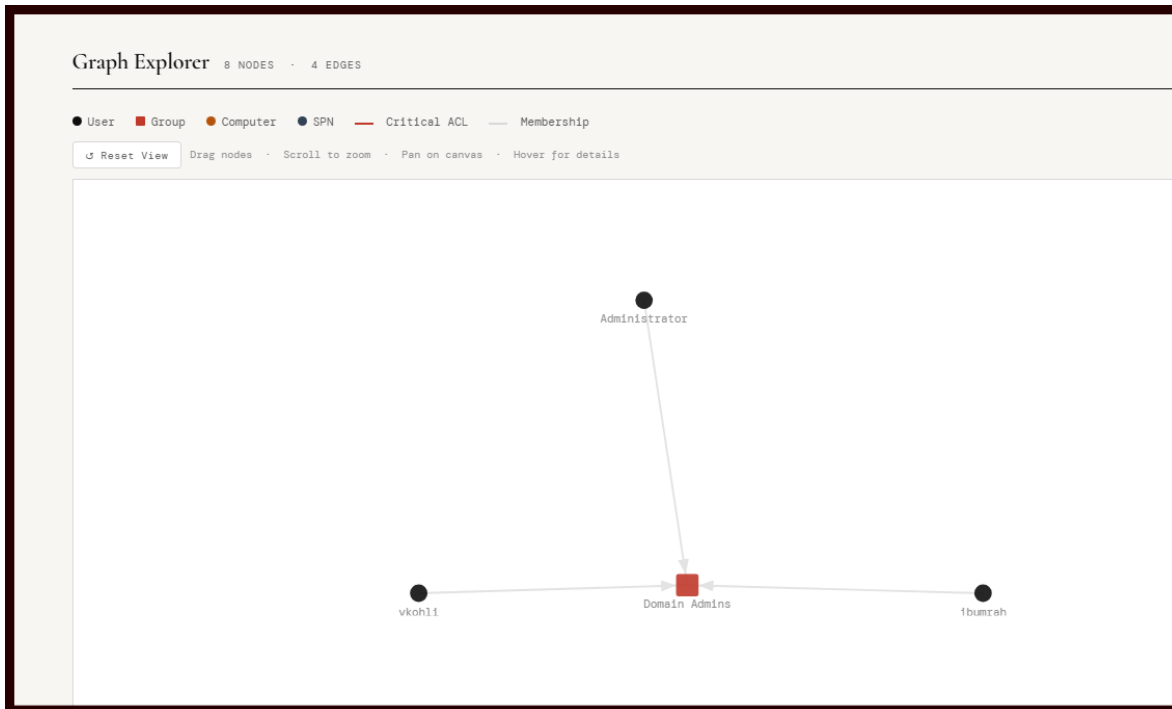
AD-AuthGraph HTML report Executive summary section

Domain Admin Privilege Paths 3 IDENTITIES

IDENTITY	SHORTEST PATH	PATHS	HOPS	PROX	OU	BFS
Administrator	Administrator → Domain Admins	172	2	1	BLUES.LOCAL // (Domain Root)	✓
vkohli	vkohli → Domain Admins	96	2	1	BLUES.LOCAL // (Domain Root)	✓
jbumrah	jbumrah → Domain Admins	96	2	1	BLUES.LOCAL // (Domain Root)	✓

Kerberosable Identities

AD-AuthGraph HTML report Domain Admin Privilege Paths



AD-AuthGraph HTML report Graph Explorer shows nodes vkohli and jbumrah path to Domain Admins

As evident from the AD-AuthGraph HTML report, that it has been able to identify the users vkohli and jbumrah have a direct path to Domain Admins.

State 2

ResetPassword allows low priv user mdsiraj Domain admin escalation

As, stated earlier in the Function vs Security protocol members of Helpdesk can be allowed to reset passwords.

Therefore, mdsiraj (member of Helpdesk group) is granted **ResetPassword** ACE on vkohli(has Domain Admin privilege as of current state).

ResetPassword is an Extended right in Active Directory.

Extended rights are special non-standard permissions which are assigned to allow specific administrative, high privilege actions on the target object. For example, ResetPassword allows the user to change password(without knowing the current one) of the target object without the requirement for broad controls like GenericAll or WriteDacl.

Powershell command to allow mdsiraj ResetPassword on vkohli

```
dsaccls "CN=vkohli,CN=Users,DC=blues,DC=local" /G "blues\mdsiraj:CA;Reset Password"
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> dsaccls "CN=vkohli,CN=Users,DC=blues,DC=local" /G "blues\mdsiraj:CA;Reset Password"
Owner: BLUES\Domain Admins
Group: BLUES\Domain Admins
```

Applied mdsiraj granted ResetPassword on vkohli

Verification

```
The command completed successfully
PS C:\Users\Administrator\AD_Authorization_Graph_project> dsaccls "CN=vkohli,CN=Users,DC=blues,DC=local" | Select-String "mdsiraj"
Allow BLUES\mdsiraj Reset Password
```

mdsiraj granted ResetPassword on vkohli, application is successful

Run the Powershell collector Collect-ADAuthorizationState.ps1 to collect the raw AD data

```
.\Collect-ADAuthorizationState.ps1 "state_2.json"
```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> .\Collect-ADAuthorizationState.ps1 "state_2.json"
[+] ActiveDirectory module loaded

=== Active Directory Authorization State Collection ===
Domain: BLUES.LOCAL
Output: state_2.json

[*] Collecting users...
    [+] Collected 11 users
[*] Collecting groups...
    [+] Collected 58 groups
[*] Collecting computers...
    [+] Collected 3 computers
[*] Collecting group memberships (including nested)...
    [+] Collected 55 group memberships
[*] Collecting Service Principal Names (SPNs)...
    [+] Collected 15 SPNs
[*] Collecting delegation configurations...
    [+] Collected 1 delegation configurations
[*] Collecting ACL permissions (this may take a while)...
    [*] Processed ACLs for 10 users...
    [*] Processed ACLs for 10 groups...
    [*] Processed ACLs for 20 groups...
    [*] Processed ACLs for 30 groups...
    [*] Processed ACLs for 40 groups...
    [*] Processed ACLs for 50 groups...

```

Collect-ADAuthorizationState.ps1 script terminal response for AD data collected of State 2

```

    [*] Processed ACLs for 50 groups...
    [+] Collected 459 ACL permissions

[*] Exporting to JSON...
    [+] Export successful: state_2.json (455.35 KB)

=== Collection Summary ===
Users:                11
Groups:               58
Computers:            3
Group Memberships:   55
SPNs:                 15
Delegations:          1
ACL Permissions:      459

[+] Collection complete!
Next step: python main.py --state state_2.json

PS C:\Users\Administrator\AD_Authorization_Graph_project> |

```

Collect-ADAuthorizationState.ps1 script terminal response for AD data collected of State 2

Execute the Python pipeline with main.py

```

python main.py .\state_2.json --output .\reports
# the ooutput files including the HTML report will be stored in a dedicated folder

```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py .\state_2.json --output .\reports\
=====
AD AUTHORIZATION GRAPH ANALYSIS PIPELINE
=====
Input: state_2.json
Output: reports

[STEP 1/4] Building Authorization Graph
=====
[*] Loading authorization state from: state_2.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 17:32:27
[+] Graph built successfully
    Nodes: 88
    Edges: 262
[+] Graph data exported to: reports\state_2_graph.json
[+] Graph exported: reports\state_2_graph.json
[+] Graph Statistics:
    Nodes:      88
    Edges:     262
    Users:      11
    Groups:     58
    Computers:  3

```

main.py executing the State 2 Python pipeline. [1/4] shows terminal output from graph_builder.py.

```

[STEP 2/4] Analyzing Authorization Graph
=====
[*] Loading graph from: reports\state_2_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 262 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 17:32:27
[+] Graph loaded: 88 nodes, 262 edges
[+] Report saved to: reports\state_2_analysis.json

[+] Analysis Summary:
    Domain Admin Paths:      4
    Kerberoastable Identities: 3
    ACL Abuse Chains:       11
    Delegation Risks:        1
[+] Analysis saved: reports\state_2_analysis.json

```

main.py executing the State 2 Python pipeline. [2/4] shows terminal output from analyzer.py.

```

[STEP 3/4] Refining Analysis (OU ACL Path Context)
-----
[*] Loading analysis: reports\state_2_analysis.json
[*] Building IdentityRegistry ...
    [+] IdentityRegistry: 88 nodes loaded from graph JSON
    [+] Tier-0 anchors identified: 12
    [+] Using REVERSE edges: 19 nodes reach Tier 0
[*] Running refinement passes ...
    [+] DA path map: 4 identities mapped
    [+] Kerberoastable OU map: 3 entries
    [+] Delegation OU map: 1 entries
    [+] OU risk heatmap: 2 distinct OU paths
[+] Refined output saved: reports\state_2_refined.json

[+] Refinement Summary:
    Registry size:          88 nodes
    ACL findings (total):   36
    High-priority ACL:      0
    Identities with DA paths: 4
    Kerberoastable:        3
    Delegation risks:       1
[+] Refined JSON saved: reports\state_2_refined.json

```

main.py executing the State 2 Python pipeline. [3/4] shows terminal output from refiner.py.

```

[STEP 4/4] Generating Interactive HTML Report
-----
[+] HTML report saved: reports\state_2_report_refined.html
    Size: 84.0 KB

[OPTIONAL] Graph PNG (SKIPPED - matplotlib not available)
    pip install matplotlib to enable

=====
PIPELINE COMPLETE
=====
Time Elapsed: 1.56 seconds

Generated Files:
1. Graph JSON:    reports\state_2_graph.json
2. Analysis JSON: reports\state_2_analysis.json
3. Refined JSON:  reports\state_2_refined.json
4. HTML Report:   reports\state_2_report_refined.html

Open report: file:///C:/Users/Administrator/AD_Authorization_Graph_project/reports/state_2_report_refined.html

```

main.py executing the State 2 Python pipeline. [4/4] shows terminal output from visualizer.py.

AD-AuthGraph output files for State 2

 state_1_report_refined	5/4/2026 8:41 PM	Microsoft Edge H...	78 KB
 state_2_analysis.json	5/6/2026 5:34 PM	JSON File	175 KB
 state_2_graph.json	5/6/2026 5:34 PM	JSON File	111 KB
 state_2_refined.json	5/6/2026 5:34 PM	JSON File	105 KB
 state_2_report_refined	5/6/2026 5:34 PM	Microsoft Edge H...	85 KB

State 2 files ouput

AD-AuthGraph HTML report generated for State 2

AD AuthGraph

BLUES.LOCAL
Collection 2026-05-06 17:32:27
Refined 2026-05-06 12:04:01
Report 2026-05-06 17:34:01

Executive Summary

ACL ABUSE CHAINS 36	DA PATH HOLDERS 4 Identities -> Domain Admins	KERBEROASTABLE 3	DELEGATION RISKS 1 configured delegations	REGISTRY SIZE 88 AD objects tracked
------------------------	---	---------------------	---	---

Report - executive summary section

mdsiraj -> vkohli -> Domain Admins

BLUES.LOCAL // (Domain Root) -> BLUES.LOCAL // (Domain Root)

Permission: ExtendedRight Chain: 3 hops Source Proximity: prox: 2 Target Proximity: prox: 1

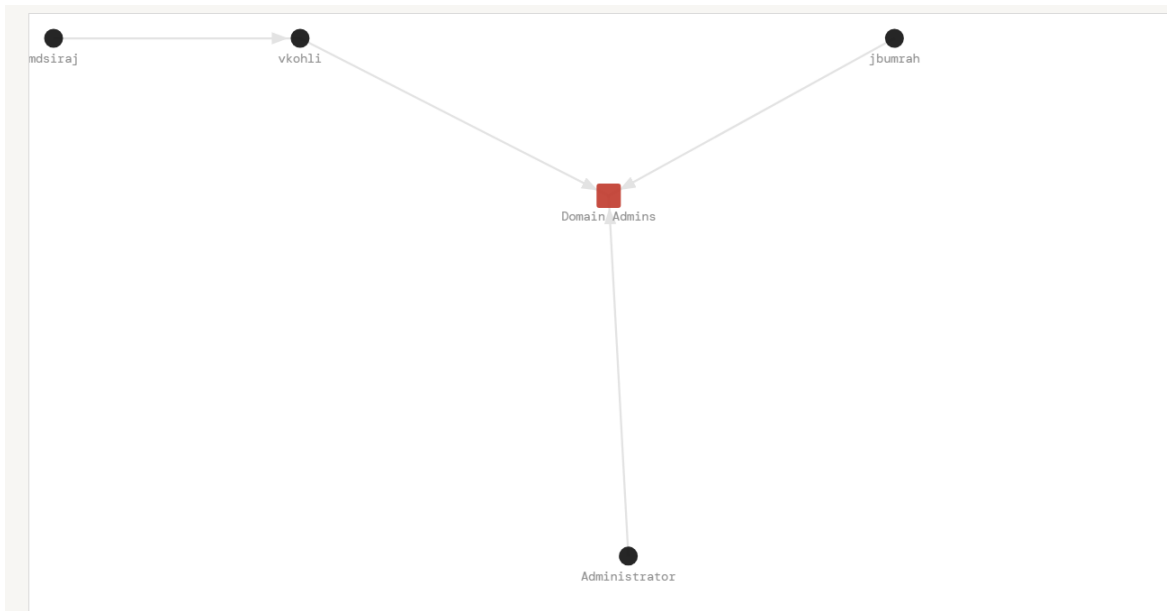
User:mdsiraj can use ResetPassword on User:vkohli to gain access to Domain Admins via: User:mdsiraj -> User:vkohli -> Group:Domain Admins

ACL abuse chains - AD-AuthGraph has detected msdiraj can use ResetPassword on vkohli

Domain Admin Privilege Paths 4 IDENTITIES						
IDENTITY	SHORTEST PATH	PATHS	HOPS	PROX	OU	BFS
Administrator	Administrator -> Domain Admins	226	2	1	BLUES.LOCAL // (Domain Root)	✓
vkohli	vkohli -> Domain Admins	96	2	1	BLUES.LOCAL // (Domain Root)	✓
jbumrah	jbumrah -> Domain Admins	123	2	1	BLUES.LOCAL // (Domain Root)	✓
mdsiraj	mdsiraj -> vkohli -> Domain Admins	96	3	2	BLUES.LOCAL // (Domain Root)	✓

Domain Admin Privilege Paths section - Analysis reveals that granting ResetPassword extended

right creates a new escalation path



Graph explorer in report visually shows mdsiraj has privilege escalation path to Domain Admins via vkohli

We see that AD-AuthGraph is able to detect and analyse that granting of the extended right ResetPassword to mdsiraj on vkohli.

State 3

The current privilege escalation chain is made longer by granting rsharma WriteDacl on mdsiraj.

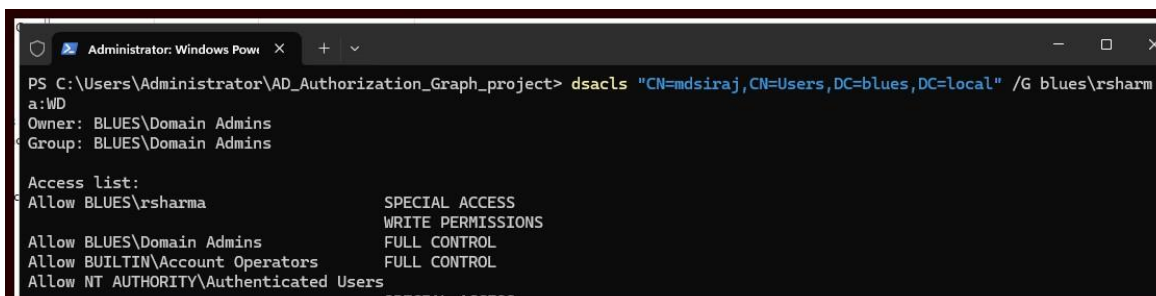
As, we have already discussed WriteDacl is a powerful permission which grants the user the ability to add, remove or modify ACE in the target object's DACL.

Therefore, the current privilege escalation $\text{mdsiraj} \rightarrow \text{vkohli} \rightarrow \text{Domain Admins}$ theoretically extends to $\text{rsharma} \rightarrow \text{mdsiraj} \rightarrow \text{vkohli} \rightarrow \text{Domain Admins}$.

We now check if AD-AuthGraph is able to detect this new authorization state and analyze it to be a genuine extension of the existing privilege escalation path.

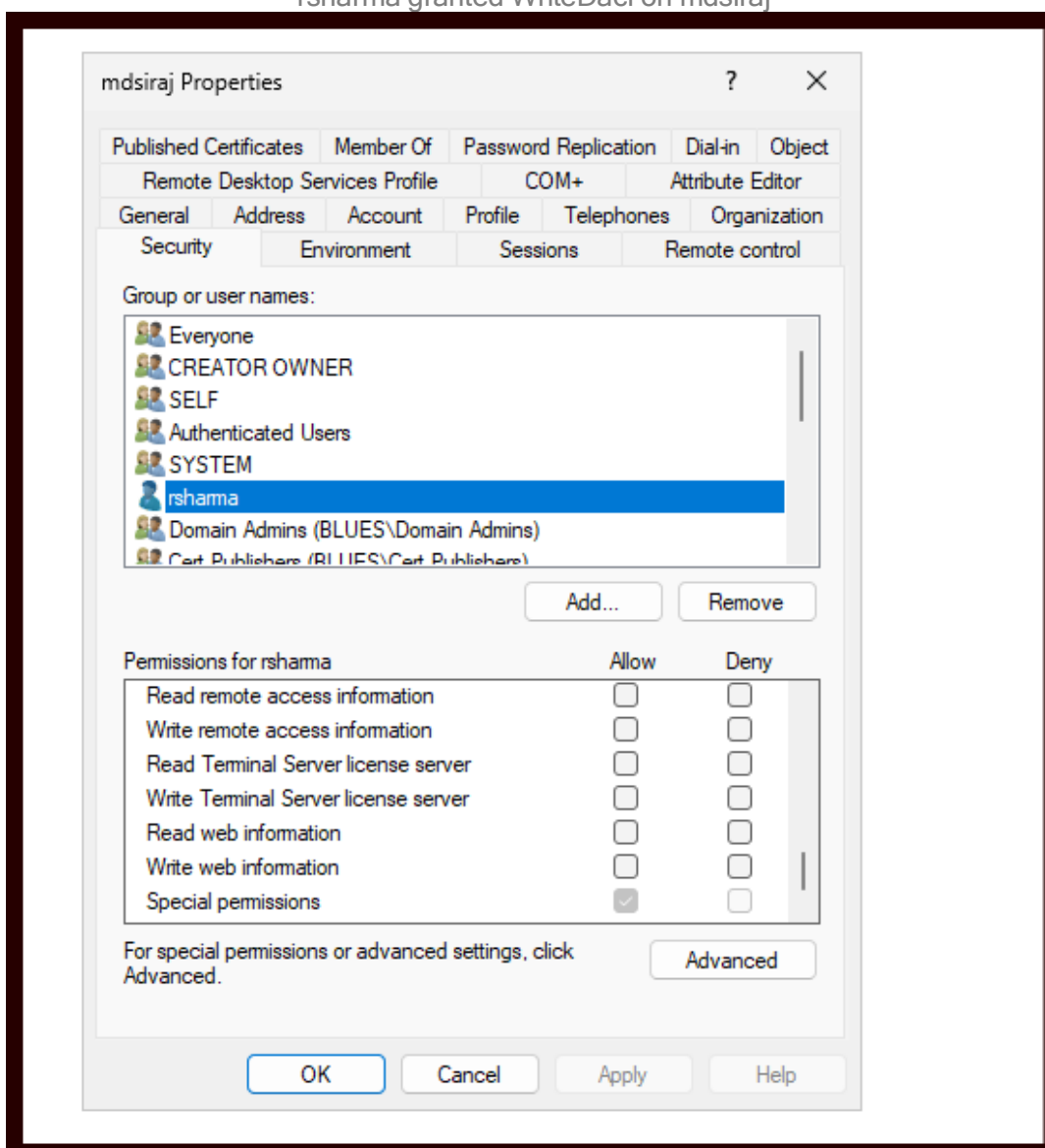
rsharma granted WriteDacl on mdsiraj

```
#Powershell command  
dscls "CN=mdsiraj,CN=Users,DC=blues,DC=local" /G blues\rsharma:WD
```



```
Administrator: Windows PowerShell  
PS C:\Users\Administrator\AD_Authorization_Graph_project> dscls "CN=mdsiraj,CN=Users,DC=blues,DC=local" /G blues\rsharma:WD  
a:WD  
Owner: BLUES\Domain Admins  
Group: BLUES\Domain Admins  
  
Access List:  
Allow BLUES\rsharma                SPECIAL ACCESS  
                                   WRITE PERMISSIONS  
Allow BLUES\Domain Admins          FULL CONTROL  
Allow BUILTIN\Account Operators    FULL CONTROL  
Allow NT AUTHORITY\Authenticated Users
```

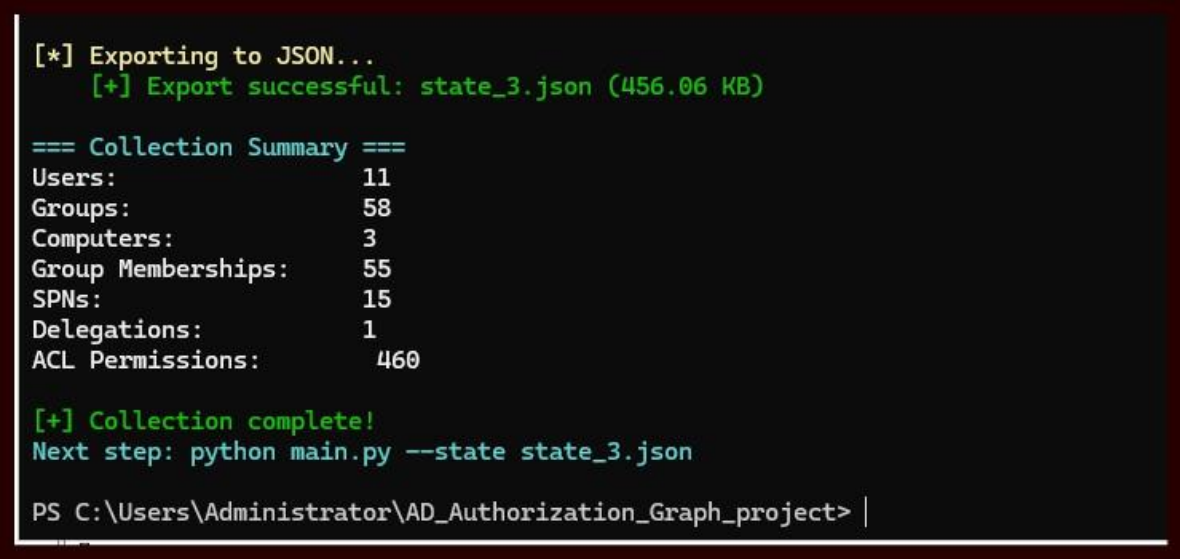
rsharma granted WriteDacL on mdsiraj



ADUC confirms that rsharma privileges over mdsiraj

Run the Powershell collector **Collect-ADAuthorizationState.ps1** to collect the raw AD data

```
.\Collect-ADAuthorizationState.ps1 "state_3.json"
```



```
[*] Exporting to JSON...  
[+] Export successful: state_3.json (456.06 KB)  
  
=== Collection Summary ===  
Users: 11  
Groups: 58  
Computers: 3  
Group Memberships: 55  
SPNs: 15  
Delegations: 1  
ACL Permissions: 460  
  
[+] Collection complete!  
Next step: python main.py --state state_3.json  
  
PS C:\Users\Administrator\AD_Authorization_Graph_project> |
```

Collect-ADAuthorizationState.ps1 script terminal output for AD data collected of State 3

Execute the Python pipeline with **main.py**

```
python main.py .\state_3.json --output .\reports  
# the output files including the HTML report will be stored in a dedicated folder
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py .\state_3.json --output .\reports\  
=====AD AUTHORIZATION GRAPH ANALYSIS PIPELINE=====
```

Input: state_3.json
Output: reports

[STEP 1/4] Building Authorization Graph

[*] Loading authorization state from: state_3.json
Domain: BLUES.LOCAL
Collection Date: 2026-05-05 15:10:49

[+] Graph built successfully
Nodes: 88
Edges: 263

[+] Graph data exported to: reports\state_3_graph.json

[+] Graph exported: reports\state_3_graph.json

[+] Graph Statistics:

Nodes:	88
Edges:	263
Users:	11
Groups:	58
Computers:	3

main.py executing the State 3 Python pipeline. [1/4] shows terminal output from graph_builder.py

```
[STEP 2/4] Analyzing Authorization Graph
```

[*] Loading graph from: reports\state_3_graph.json

[+] Loaded 88 nodes

[+] Loaded 263 edges

Domain: BLUES.LOCAL
Collection Date: 2026-05-05 15:10:49

[+] Graph loaded: 88 nodes, 263 edges

[+] Report saved to: reports\state_3_analysis.json

[+] Analysis Summary:

Domain Admin Paths:	5
Kerberoastable Identities:	3
ACL Abuse Chains:	12
Delegation Risks:	1

[+] Analysis saved: reports\state_3_analysis.json

main.py executing the State 3 Python pipeline. [2/4] shows terminal output from analyzer.py

```

[STEP 3/4] Refining Analysis (OU ACL Path Context)
-----
[*] Loading analysis: reports\state_3_analysis.json
[*] Building IdentityRegistry ...
    [+] IdentityRegistry: 88 nodes loaded from graph JSON
    [+] Tier-0 anchors identified: 12
    [+] Using REVERSE edges: 20 nodes reach Tier 0
[*] Running refinement passes ...
    [+] DA path map: 5 identities mapped
    [+] Kerberoastable OU map: 3 entries
    [+] Delegation OU map: 1 entries
    [+] OU risk heatmap: 2 distinct OU paths
[+] Refined output saved: reports\state_3_refined.json

[+] Refinement Summary:
    Registry size:      88 nodes
    ACL findings (total): 40
    High-priority ACL:  0
    Identities with DA paths: 5
    Kerberoastable:     3
    Delegation risks:   1
[+] Refined JSON saved: reports\state_3_refined.json

```

main.py executing the State 3 Python pipeline. [3/4] shows terminal output from refiner.py.

```

[STEP 4/4] Generating Interactive HTML Report
-----
[+] HTML report saved: reports\state_3_report_refined.html
    Size: 90.8 KB

[OPTIONAL] Graph PNG (SKIPPED - matplotlib not available)
    pip install matplotlib to enable

=====
PIPELINE COMPLETE
=====
Time Elapsed: 1.61 seconds

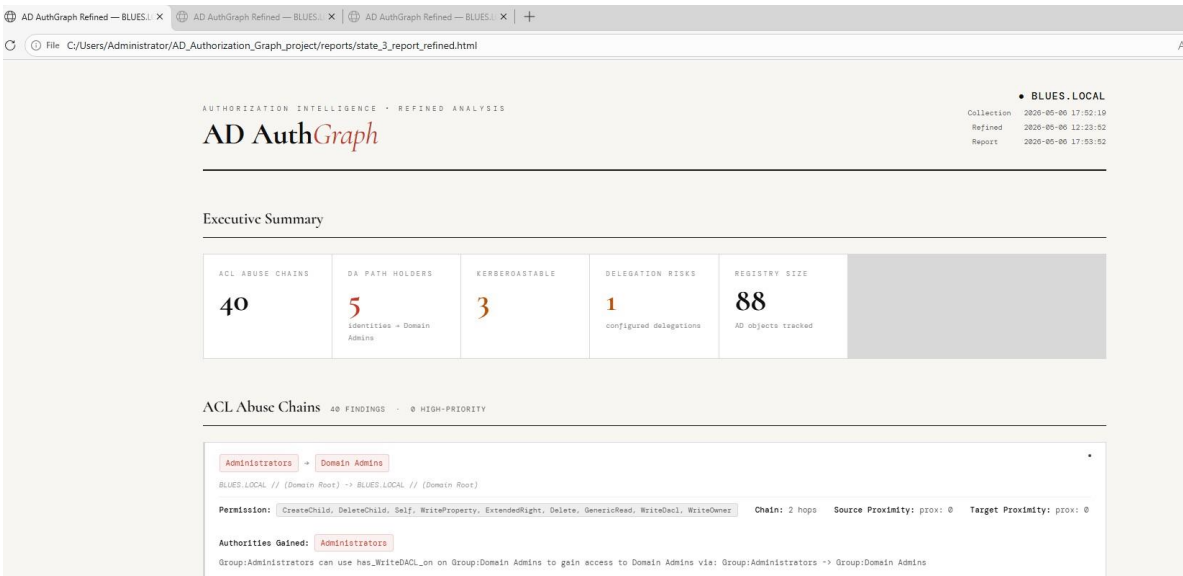
Generated Files:
  1. Graph JSON:    reports\state_3_graph.json
  2. Analysis JSON: reports\state_3_analysis.json
  3. Refined JSON:  reports\state_3_refined.json
  4. HTML Report:   reports\state_3_report_refined.html

Open report: file:///C:/Users/Administrator/AD_Authorization_Graph_project/reports/state_3_report_refined.html

```

main.py executing the State 3 Python pipeline. [4/4] shows terminal output from visualizer.py.

AD-AuthGraph HTML report generated for State 3



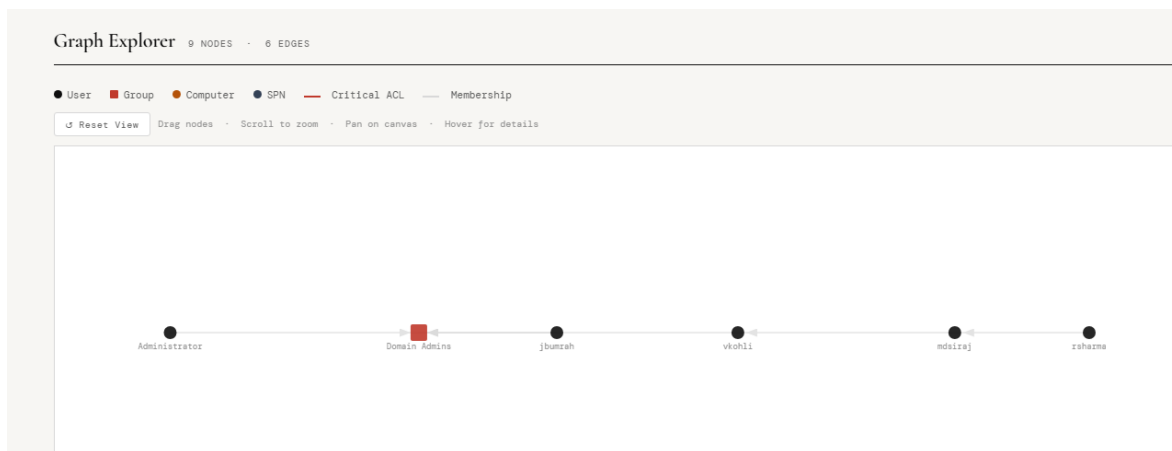
Report - executive summary section



ACL abuse chains - AD-AuthGraph has detected rsharma has WriteDacl on mdsiraj

Domain Admin Privilege Paths 5 IDENTITIES						
IDENTITY	SHORTEST PATH		PATHS	HOPS	PROX	BFS
Administrator	Administrator -> Domain Admins		280	2	1	✓
vkohl1	vkohl1 -> Domain Admins		96	2	1	✓
jbumrah	jbumrah -> Domain Admins		150	2	1	✓
mdsiraj	mdsiraj -> vkohl1 -> Domain Admins		96	3	2	✓
rsharma	rsharma -> mdsiraj -> vkohl1 -> Domain Admins		96	4	3	✓

Domain Admin Privilege Paths section - Analysis reveals that granting WriteDacl creates a new escalation path



Graph explorer in report visually shows the privilege escalation path extended to rsharma

AD-AuthGraph not only detects the new authorization state but also is able to analyse and visually represent the new escalation path created:

rsharma → *mdsraj* → *vkoht* → *Domain Admins*

State 4

As per the Security vs Function protocol a member of the custom group IT can be granted ResetPassword extended right on other user objects.

rjadeja is granted ResetPassword on rsharma.

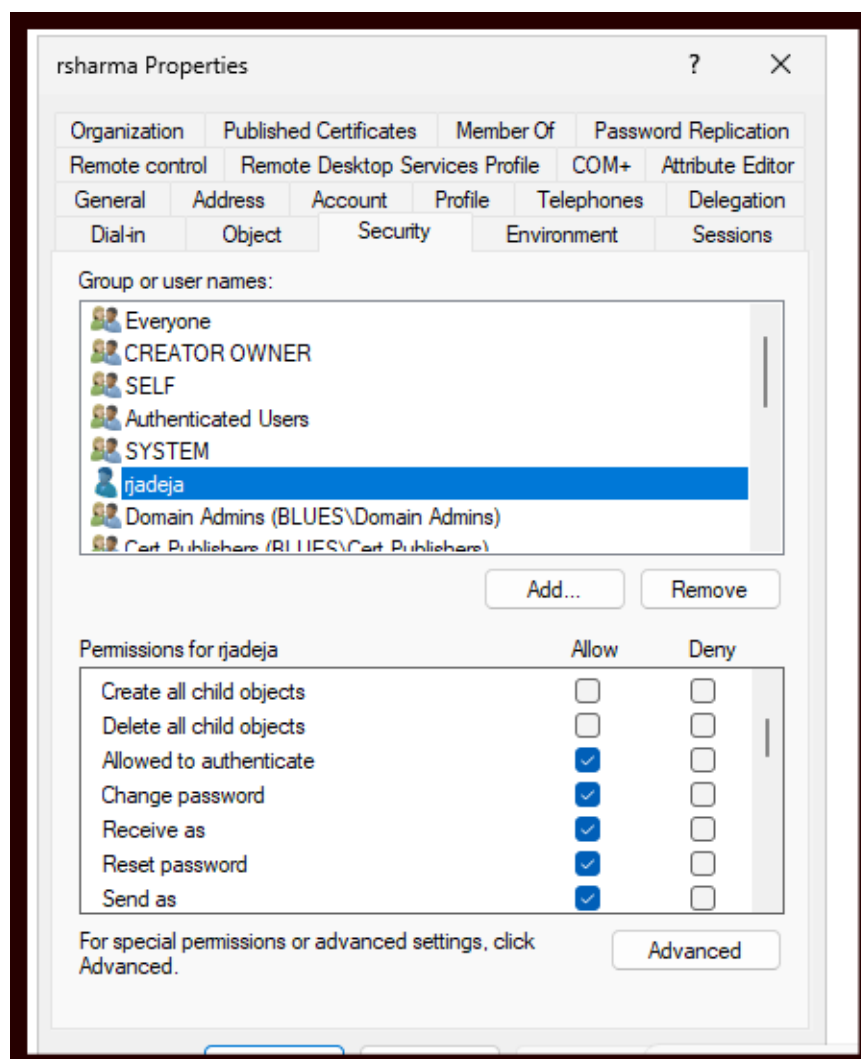
We shall now see that AD-Authgraph is able to detect this new authorization state, and analyse this as an extension of the potential privilege escalation chain

rsharma → *mdsj* → *vkoh* → *Domain Admins*.

Powershell command to grant rjadeja ResetPassword on rsharma.

```
dsaccls "CN=rsharma,CN=Users,DC=blues,DC=local" /G "blues\rjadeja:CA;Reset Password"
```

Verification



ADUC confirms rjadeja has privileges over rsharma

Run the Powershell collector Collect-ADAuthorizationState.ps1 to collect the raw AD data

```
.\Collect-ADAuthorizationState.ps1 "state_4.json"
```

Execute the Python pipeline with main.py

```
python main.py .\state_4.json --output .\reports
# the output files including the HTML report will be stored in a dedicated folder
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py .\state_4.json --output .\reports\
=====
AD AUTHORIZATION GRAPH ANALYSIS PIPELINE
=====
Input: state_4.json
Output: reports

[STEP 1/4] Building Authorization Graph
=====
[*] Loading authorization state from: state_4.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57
[+] Graph built successfully
    Nodes: 88
    Edges: 264
[+] Graph data exported to: reports\state_4_graph.json
[+] Graph exported: reports\state_4_graph.json
[+] Graph Statistics:
    Nodes:      88
    Edges:     264
    Users:      11
    Groups:     58
    Computers:  3
```

main.py executing the State 4 Python pipeline. [1/4] shows terminal output from graph_builder.py

```
[STEP 2/4] Analyzing Authorization Graph
=====
[*] Loading graph from: reports\state_4_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 264 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57
[+] Graph loaded: 88 nodes, 264 edges
[+] Report saved to: reports\state_4_analysis.json

[+] Analysis Summary:
    Domain Admin Paths:      6
    Kerberoastable Identities: 3
    ACL Abuse Chains:       13
    Delegation Risks:        1
[+] Analysis saved: reports\state_4_analysis.json
```

main.py executing the State 4 Python pipeline. [2/4] shows terminal output from graph_builder.py

```
[STEP 4/4] Generating Interactive HTML Report
-----
[+] HTML report saved: reports\state_4_report_refined.html
    Size: 98.1 KB

[OPTIONAL] Graph PNG (SKIPPED - matplotlib not available)
    pip install matplotlib to enable

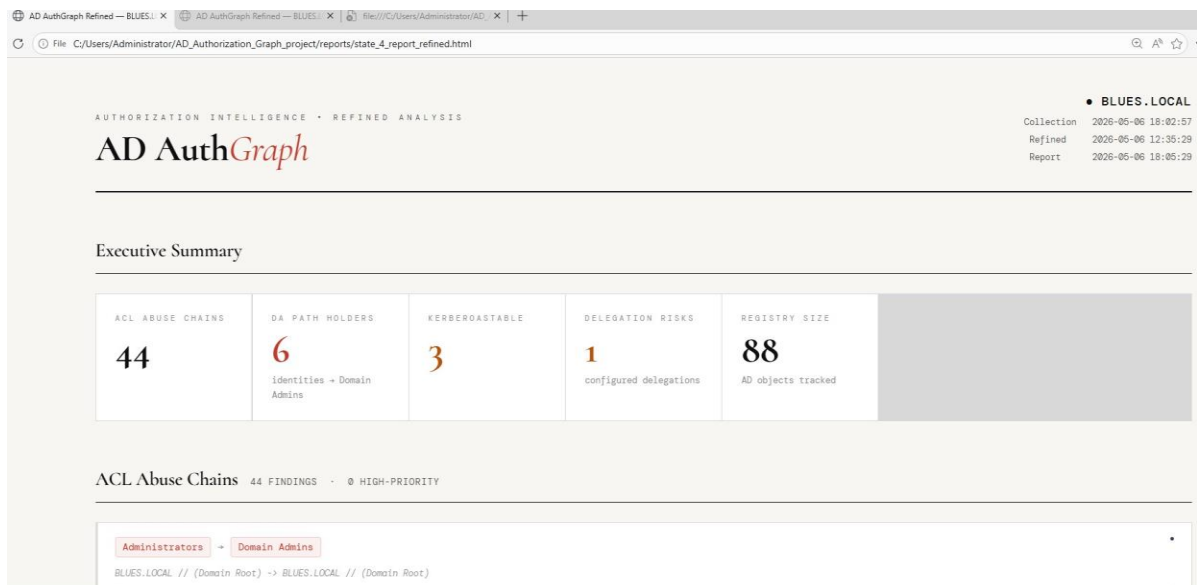
=====
PIPELINE COMPLETE
=====
Time Elapsed: 1.88 seconds

Generated Files:
1. Graph JSON:    reports\state_4_graph.json
2. Analysis JSON: reports\state_4_analysis.json
3. Refined JSON:  reports\state_4_refined.json
4. HTML Report:  reports\state_4_report_refined.html

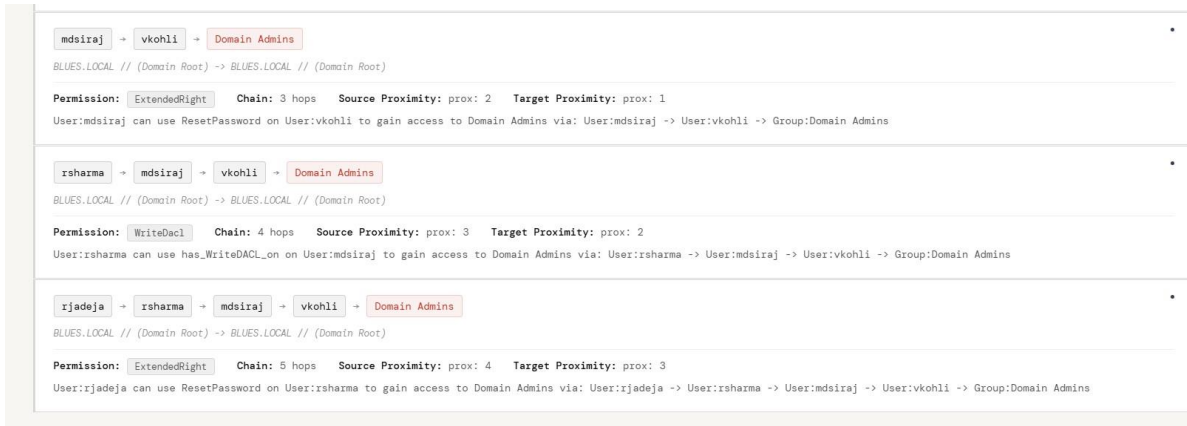
Open report: file:///C:/Users/Administrator/AD_Authorization_Graph_project/reports/state_4_report_refined.html
PS C:\Users\Administrator\AD_Authorization_Graph_project>
```

main.py executing the State 4 Python pipeline. [4/4] shows terminal output from graph_builder.py and confirmation generation of the output files

AD-AuthGraph HTML report generated for State 4



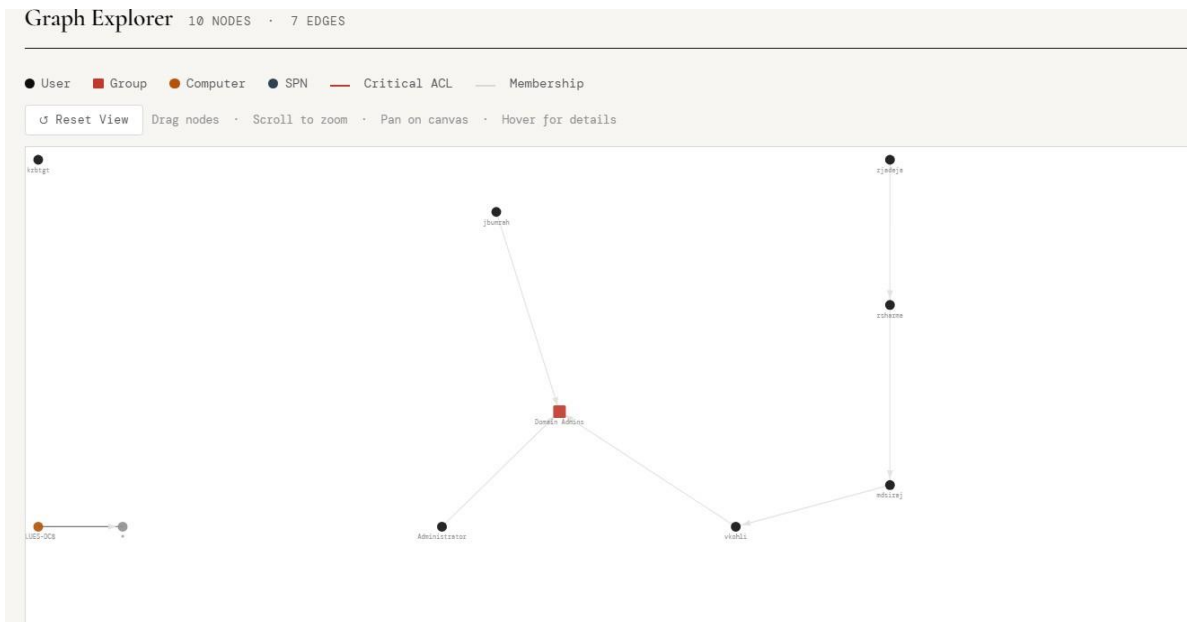
Report - executive summary section



Domain Admin Privilege Paths section - Analysis reveals that granting ResetPassword to rjadeja on rsharma creates a new escalation path

Kerberoastable Identities		3 IDENTITIES	0 HIGH-PRIORITY
rsharma	MEDIUM	DA PATH	ENABLED
HTTP/hr-app.blues.local			
OU: BLUES.LOCAL // (Domain Root) SPN Count: 1 DA Paths: 96			
krbtgt	MEDIUM	DISABLED	
kadmin/changepw			
OU: BLUES.LOCAL // (Domain Root) SPN Count: 1 DA Paths: 0			
BLUES-DC\$	MEDIUM	ENABLED	
Dfsr-12F9A27C-BF97-4787-9364-D31B6C55EB04/Blues-DC.Blues.local ldap/Blues-DC.Blues.local/ForestDnsZones.Blues.local ldap/Blues-DC.Blues.local/DomainDnsZones.Blues.local DNS/Blues-DC.Blues.local GC/Blues-DC.Blues.local/Blues.local RPC/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b_msdcs.Blues.local E3514235-4B06-11D1-AB04-00C04FC2DCD2/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b/Blues.local ldap/BLUES-DC/BLUES ldap/2e7cb30a-bcf3-4cf2-9285-5dcf28936f2b_msdcs.Blues.local ldap/Blues-DC.Blues.local/BLUES ldap/BLUES-DC ldap/Blues-DC.Blues.local ldap/Blues-DC.Blues.local/Blues.local			
OU: BLUES.LOCAL // Domain Controllers SPN Count: 13 DA Paths: 0			

rsharma is possibly susceptible to kerberoasting



Graph explorer shows privilege escalation path due to the new authorization state

AD-AuthGraph detects the new authorization state due to ResetPassword granted to rjadeja on rsharma. It is also able to analyze that there is an extension of the privilege escalation to Domain Admins chain:

rjadeja → rsharma → mdsaj → vkoh → Domain Admins

State 5

In this state we will see the implication of granting the WriteProperty authorization to user rashwin on Director (nested in Domain Admins) group.

rashwin is a standard user and has no Domain Admin privilege.

The custom group Director is member of the built-in group Domain Admins.

rashwin granted WriteProperty on Director group

```
#Powershell command  
dsacl "CN=Director,CN=Users,DC=blues,DC=local" /G blues\rashwin:WP
```

Verification

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> dscls "CN=Director,CN=Users,DC=blues,DC=local" | Select-String "rashwin"

Allow BLUES\rashwin          SPECIAL ACCESS
```

rashwin granted WriteProperty on custom group Director

Run the Powershell collector Collect-ADAuthorizationState.ps1 to collect the raw AD data

```
.\Collect-ADAuthorizationState.ps1 "state_5.json"
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> .\Collect-ADAuthorizationState.ps1 "state_5.json"
[+] ActiveDirectory module loaded

=== Active Directory Authorization State Collection ===
Domain: BLUES.LOCAL
Output: state_5.json

[*] Collecting users...
    [+] Collected 11 users
[*] Collecting groups...
    [+] Collected 58 groups
[*] Collecting computers...
    [+] Collected 3 computers
[*] Collecting group memberships (including nested)...
    [+] Collected 55 group memberships
[*] Collecting Service Principal Names (SPNs)...
    [+] Collected 15 SPNs
[*] Collecting delegation configurations...
    [+] Collected 1 delegation configurations
[*] Collecting ACL permissions (this may take a while)...
    [*] Processed ACLs for 10 users...
    [*] Processed ACLs for 10 groups...
    [*] Processed ACLs for 20 groups...
    [*] Processed ACLs for 30 groups...
    [*] Processed ACLs for 40 groups...
    [*] Processed ACLs for 50 groups...
    [+] Collected 462 ACL permissions
```

Collect-ADAuthorizationState.ps1 script terminal output for AD data collected of State 5

```

[*] Exporting to JSON...
    [+] Export successful: state_5.json (457.68 KB)

=== Collection Summary ===
Users:          11
Groups:         58
Computers:      3
Group Memberships: 55
SPNs:           15
Delegations:    1
ACL Permissions: 462

[+] Collection complete!
Next step: python main.py --state state_5.json

```

Collect-ADAuthorizationState.ps1 script terminal output for AD data collected of State 5, raw data collected.

Execute the Python pipeline with main.py

```

python main.py .\state_5.json --output .\reports
# the output files including the HTML report will be stored in a dedicated folder

```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py .\state_5.json --output .\reports\
=====
AD AUTHORIZATION GRAPH ANALYSIS PIPELINE
=====
Input: state_5.json
Output: reports

[STEP 1/4] Building Authorization Graph
=====
[*] Loading authorization state from: state_5.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
[+] Graph built successfully
    Nodes: 88
    Edges: 265
[+] Graph data exported to: reports\state_5_graph.json
[+] Graph exported: reports\state_5_graph.json
[+] Graph Statistics:
    Nodes: 88
    Edges: 265
    Users: 11
    Groups: 58
    Computers: 3

```

_main.py executing the State 5 Python pipeline. [1/4] shows terminal output from graph_builder.py

```

[STEP 2/4] Analyzing Authorization Graph
-----
[*] Loading graph from: reports\state_5_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 265 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
[+] Graph loaded: 88 nodes, 265 edges
[+] Report saved to: reports\state_5_analysis.json

[+] Analysis Summary:
    Domain Admin Paths:      7
    Kerberoastable Identities: 3
    ACL Abuse Chains:       14
    Delegation Risks:       1
[+] Analysis saved:  reports\state_5_analysis.json

[STEP 3/4] Refining Analysis (OU ACL Path Context)

```

main.py executing the State 5 Python pipeline. [1/4] shows terminal output from analyzer.py

```

[STEP 3/4] Refining Analysis (OU ACL Path Context)
-----
[*] Loading analysis: reports\state_5_analysis.json
[*] Building IdentityRegistry ...
    [+] IdentityRegistry: 88 nodes loaded from graph JSON
    [+] Tier-0 anchors identified: 12
    [+] Using REVERSE edges: 22 nodes reach Tier 0
[*] Running refinement passes ...
    [+] DA path map: 7 identities mapped
    [+] Kerberoastable OU map: 3 entries
    [+] Delegation OU map: 1 entries
    [+] OU risk heatmap: 2 distinct OU paths
[+] Refined output saved: reports\state_5_refined.json

[+] Refinement Summary:
    Registry size:      88 nodes
    ACL findings (total): 48
    High-priority ACL:  0
    Identities with DA paths: 7
    Kerberoastable:     3
    Delegation risks:   1
[+] Refined JSON saved: reports\state_5_refined.json

[STEP 4/4] Generating Interactive HTML Report

```

main.py executing the State 5 Python pipeline. [3/4] shows terminal output from refiner.py.

```

[STEP 4/4] Generating Interactive HTML Report
-----
[+] HTML report saved: reports\state_5_report_refined.html
    Size: 104.8 KB

[OPTIONAL] Graph PNG (SKIPPED - matplotlib not available)
           pip install matplotlib to enable

=====
PIPELINE COMPLETE
=====
Time Elapsed: 2.13 seconds

Generated Files:
  1. Graph JSON:    reports\state_5_graph.json
  2. Analysis JSON: reports\state_5_analysis.json
  3. Refined JSON:  reports\state_5_refined.json
  4. HTML Report:   reports\state_5_report_refined.html

Open report: file:///C:/Users/Administrator/AD_Authorization_Graph_project/reports/state_5_report_refined.html

```

main.py executing the State 5 Python pipeline. [4/4] shows terminal output from visualizer.py.

AD-AuthGraph HTML report generated for State 5

The screenshot shows a web browser window displaying the AD AuthGraph HTML report. The browser's address bar shows the file path: `C:/Users/Administrator/AD_Authorization_Graph_project/reports/state_5_report_refined.html`. The report header includes the title "AD AuthGraph" and a sub-header "AUTHORIZATION INTELLIGENCE • REFINED ANALYSIS". On the right, there is a table showing the report's history:

	Collection	Refined	Report
• BLUES.LOCAL	2020-05-08 18:28:11	2020-05-08 13:01:00	2020-05-08 18:31:00

The "Executive Summary" section contains a table with five metrics:

ACL ABUSE CHAINS	DA PATH HOLDERS	KERBEROASTABLE	DELEGATION RISKS	REGISTRY SIZE
48	7 Identities → Domain Admins	3	1 configured delegations	88 AD objects tracked

Below the executive summary, the "ACL Abuse Chains" section is visible, showing 48 findings and a high-priority status. It includes a diagram showing the relationship between "Administrators" and "Domain Admins" in the "BLUES.LOCAL // (Domain Root)" context.

Report - executive summary section

reshwin → Director → Domain Admins

BLUES.LOCAL // (Domain Root) → BLUES.LOCAL // (Domain Root)

Permission: WriteProperty

Chain: 3 hops

Source Proximity: prox: 2

Target Proximity: prox: 1

User:reshwin can use has_WriteProperty_on on Group:Director to gain access to Domain Admins via: User:reshwin → Group:Director → Group:Domain Admins

mdsiraaj → vkohl1 → Domain Admins

BLUES.LOCAL // (Domain Root) → BLUES.LOCAL // (Domain Root)

Permission: ExtendedRight

Chain: 3 hops

Source Proximity: prox: 2

Target Proximity: prox: 1

User:mdsiraaj can use ResetPassword on User:vkohl1 to gain access to Domain Admins via: User:mdsiraaj → User:vkohl1 → Group:Domain Admins

rsharma → mdsiraaj → vkohl1 → Domain Admins

BLUES.LOCAL // (Domain Root) → BLUES.LOCAL // (Domain Root)

Permission: WriteDACL

Chain: 4 hops

Source Proximity: prox: 3

Target Proximity: prox: 2

User:rsharma can use has_WriteDACL_on on User:mdsiraaj to gain access to Domain Admins via: User:rsharma → User:mdsiraaj → User:vkohl1 → Group:Domain Admins

rjadeja → rsharma → mdsiraaj → vkohl1 → Domain Admins

BLUES.LOCAL // (Domain Root) → BLUES.LOCAL // (Domain Root)

Permission: ExtendedRight

Chain: 5 hops

Source Proximity: prox: 4

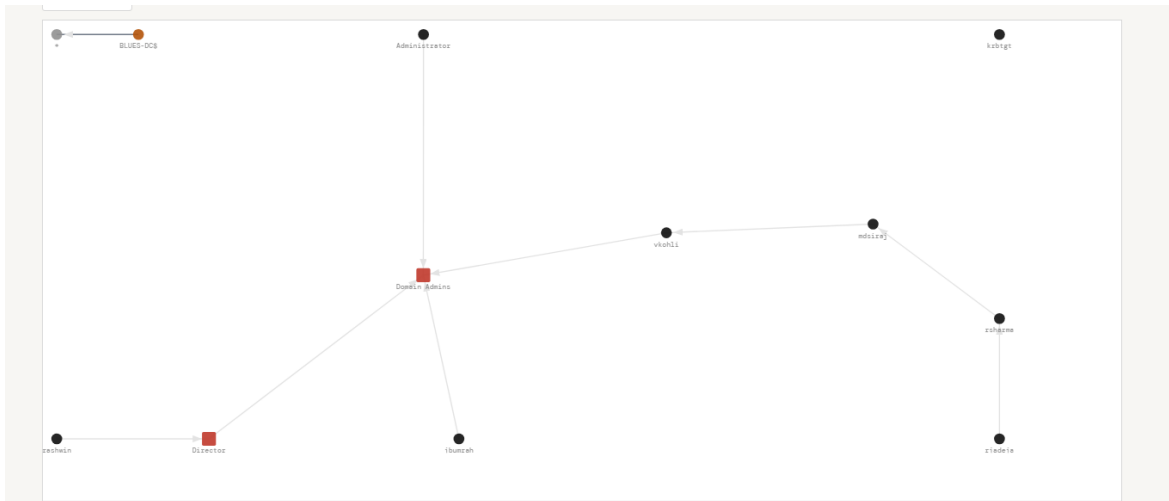
Target Proximity: prox: 3

User:rjadeja can use ResetPassword on User:rsharma to gain access to Domain Admins via: User:rjadeja → User:rsharma → User:mdsiraaj → User:vkohl1 → Group:Domain Admins

ACL abuse chains - AD-AuthGraph has detected that rashwin granted WriteProperty on Director group

IDENTITY	SHORTEST PATH	PATHS	HOPS	PROX	OU	BFS
Administrator	Administrator → Domain Admins	353	2	1	BLUES.LOCAL // (Domain Root)	✓
vkohl1	vkohl1 → Domain Admins	105	2	1	BLUES.LOCAL // (Domain Root)	✓
jbumrah	jbumrah → Domain Admins	182	2	1	BLUES.LOCAL // (Domain Root)	✓
reshwin	reshwin → Director → Domain Admins	73	3	2	BLUES.LOCAL // (Domain Root)	✓
mdsiraaj	mdsiraaj → vkohl1 → Domain Admins	105	3	2	BLUES.LOCAL // (Domain Root)	✓
rsharma	rsharma → mdsiraaj → vkohl1 → Domain Admins	105	4	3	BLUES.LOCAL // (Domain Root)	✓
rjadeja	rjadeja → rsharma → mdsiraaj → vkohl1 → Domain Admins	95	5	4	BLUES.LOCAL // (Domain Root)	✓

Domain Admin Privilege Paths section - Analysis reveals that granting the WriteProperty creates a new escalation path



Graph explorer in report visually shows the new privilege escalation path created due rashwin being granted WriteProperty on Director group

AD-AuthGraph is able to detect user on group permissions. It clearly maps that the WriteProperty granted to user rashwin on group Director creates a potential new privilege escalation path:

rashwin → Director → Domain Admins #As Director group is nested in Domain Admins group

State Comparison & Security Evolution

A comparison feature is also coded in `main.py`. The `--compare` parameter runs `graphbuilder.py` for the two states and responds with an output json file "state_comparison". "state_comparison" mainly records three types of data for both states, from the `graph_builder.py` output (i) Paths to domain admin (ii) total edges (iii) kerberoastable identity and tracks the changes while also identifying new paths or removed paths to Domain Admins.

Command structure to use the Compare feature of `main.py`

```
python main.py --compare state_x.json state_y.json --output .\output_directory\

# y has to be any previous state to x
```

```
#Example:
python main.py --compare state_5.json state_3.json --output .\deltas\
```

Expected output files(based on example state_5 vs state_3):

- state_3_graph.json
- state_5_graph.json
- state_comparison.json

Note: state_comparison.json gets overwritten every time a comparison is done.
Incase, you want to refer to a comparison state it's better to make copy.

State 1 vs State 0

Comparison

PowerShell command

```
python main.py --compare state_1.json state_0.json --output .\deltas\
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python main.py --compare state_1.json state_0.json --output .\deltas\
=====
AUTHORIZATION STATE COMPARISON
=====
Current State: state_1.json
Previous State: state_0.json

[*] Building graphs ...
[*] Loading authorization state from: state_1.json
Domain: BLUES.LOCAL
Collection Date: 2026-05-04 18:35:19
[+] Graph built successfully
Nodes: 88
Edges: 261
[+] Graph data exported to: deltas\state_1_graph.json
[*] Loading authorization state from: state_0.json
Domain: Blues.local
Collection Date: 2026-05-04 17:07:22
[+] Graph built successfully
Nodes: 88
Edges: 252
[+] Graph data exported to: deltas\state_0_graph.json
[*] Analyzing states ...
[*] Loading graph from: deltas\state_1_graph.json
[+] Loaded 88 nodes
[+] Loaded 261 edges
Domain: BLUES.LOCAL
```

Terminal output_main.py orchestrates comparison execution - graph_builder.py processes both states

```
[+] Loaded 261 edges
Domain: BLUES.LOCAL
Collection Date: 2026-05-04 18:35:19
[+] Graph loaded: 88 nodes, 261 edges
[*] Loading graph from: deltas\state_0_graph.json
[+] Loaded 88 nodes
[+] Loaded 252 edges
Domain: Blues.local
Collection Date: 2026-05-04 17:07:22
[+] Graph loaded: 88 nodes, 252 edges
[*] Comparing states ...
```

```
=====
COMPARISON RESULTS
=====
```

```
Current State (state_1.json):
Domain Admin Paths: 3
Total Edges: 261
Kerberoastable: 3
```

```
Previous State (state_0.json):
Domain Admin Paths: 1
Total Edges: 252
Kerberoastable: 3
```

```
Deltas:
DA Paths: +2
Edges: +9
```

Terminal output displaying comparison results

```
=====
COMPARISON RESULTS
=====

Current State (state_1.json):
  Domain Admin Paths: 3
  Total Edges: 261
  Kerberoastable: 3

Previous State (state_0.json):
  Domain Admin Paths: 1
  Total Edges: 252
  Kerberoastable: 3

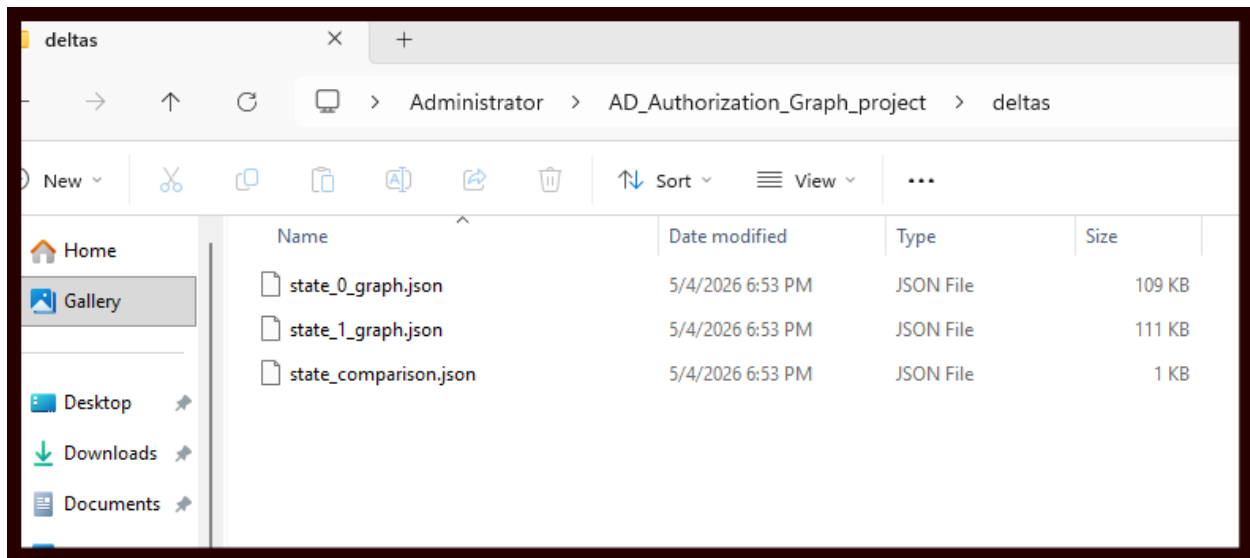
Deltas:
  DA Paths: +2
  Edges: +9
  Kerberoastable: +0

New DA Paths (2):
  + User:jbumrah
  + User:vkohli

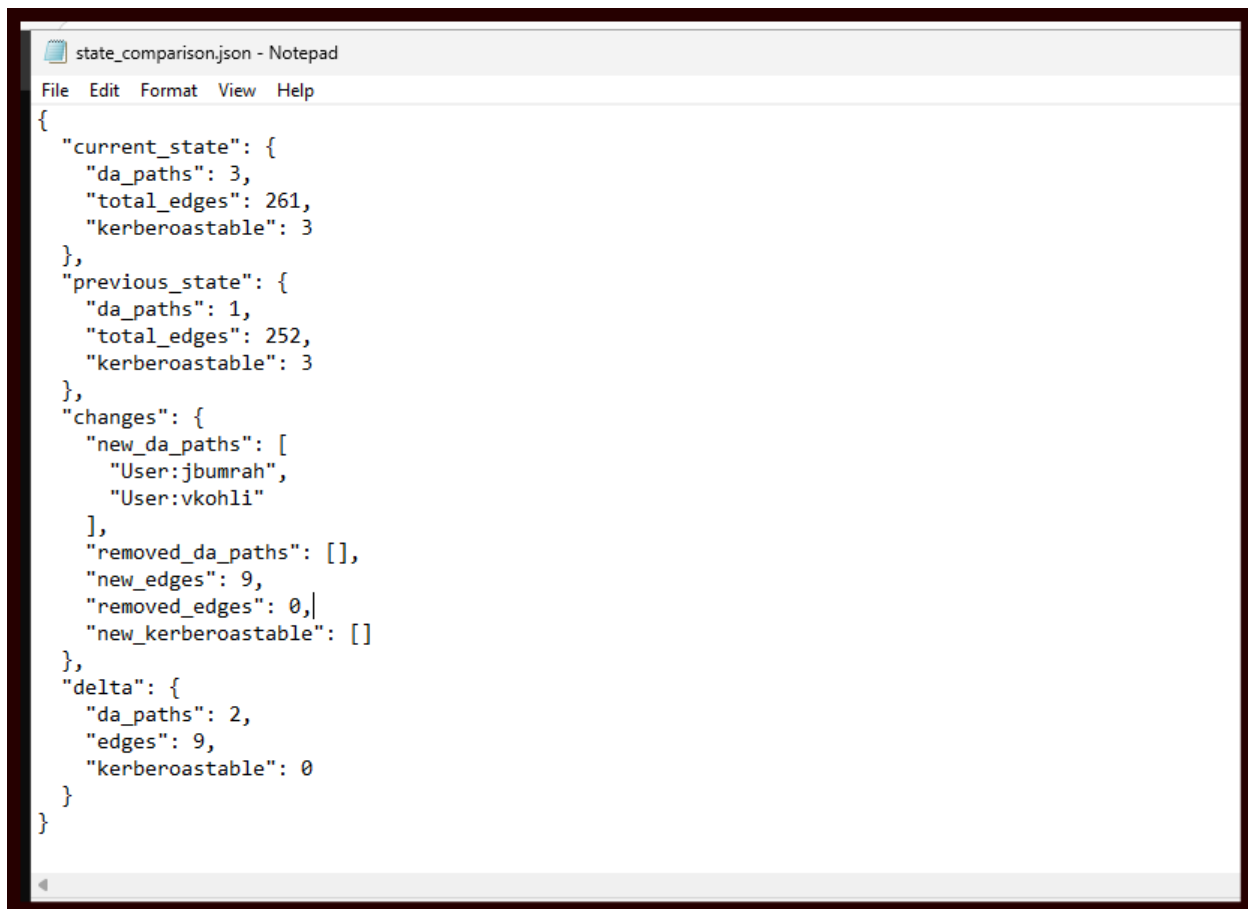
[+] Comparison saved: deltas\state_comparison.json

PS C:\Users\Administrator\AD_Authorization_Graph_project>
```

Terminal output confirms that the comparison results stored in state_comparison.json



Output files generated



```
state_comparison.json - Notepad
File Edit Format View Help
{
  "current_state": {
    "da_paths": 3,
    "total_edges": 261,
    "kerberoastable": 3
  },
  "previous_state": {
    "da_paths": 1,
    "total_edges": 252,
    "kerberoastable": 3
  },
  "changes": {
    "new_da_paths": [
      "User:jbumrah",
      "User:vkohli"
    ],
    "removed_da_paths": [],
    "new_edges": 9,
    "removed_edges": 0,
    "new_kerberoastable": []
  },
  "delta": {
    "da_paths": 2,
    "edges": 9,
    "kerberoastable": 0
  }
}
```

Data stored in state_comparison.json

Security evolution

In State 0 the custom users were not assigned any privileges, just group memberships. No, major ACL permission was also applied. As we had seen in the State 0 HTML report, the only object with a path to Domain Admins (DA) was the Administrator of the Domain Controller machine.

In State 1 the ACLs were applied. Importantly the Director group was made a member of Domain Admins, this elevated the members of Director i.e. vkohli and jbumrah to Domain Admin status.

The comparison output records this change and notes the addition of two possible escalation paths to Domain Admins, both as a result of inheriting privilege due to group nesting

vkohli → DA

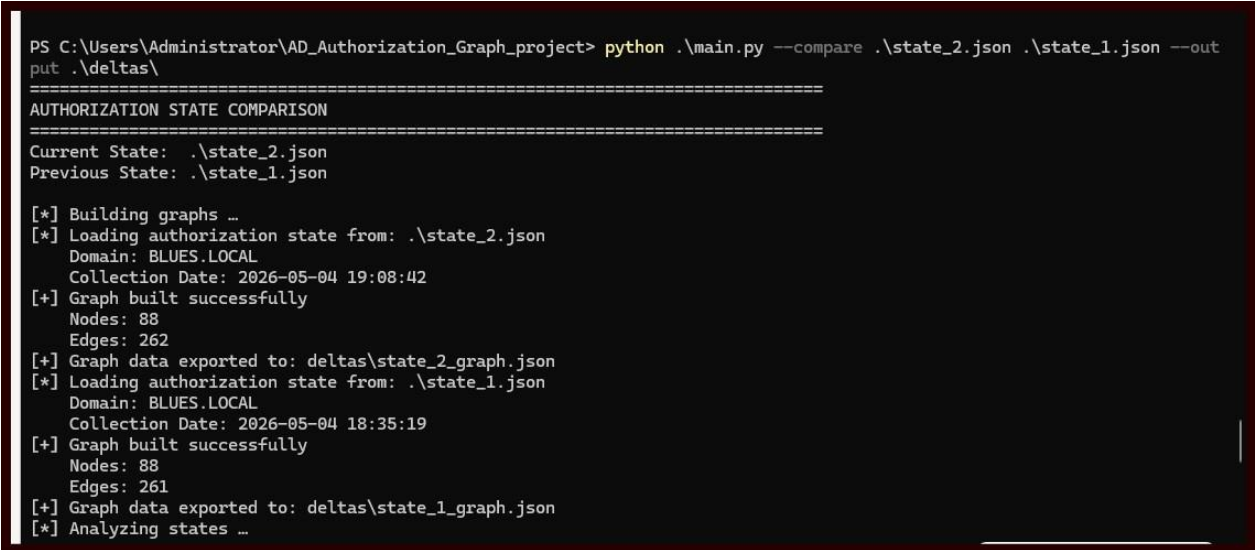
jbumrah → DA

State 2 vs State 1

User mdsiraj has WriteProperty permission and membership of the Helpdesk group, and was granted the Extended right ResetPassword on vkohli. This is defined as State 2 authorization state.

Powershell command

```
python main.py --compare state_.json state_1.json --output .\deltas\
```

A terminal window with a black background and white text. The prompt is 'PS C:\Users\Administrator\AD_Authorization_Graph_project>'. The command entered is 'python .\main.py --compare .\state_2.json .\state_1.json --output .\deltas\'. The output shows a comparison of two authorization states. It starts with 'AUTHORIZATION STATE COMPARISON' and lists the current and previous states. It then shows the process of building graphs for each state, including loading the authorization state, domain, collection date, and the number of nodes and edges. Finally, it shows the graph data being exported to JSON files in the 'deltas' directory.

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py --compare .\state_2.json .\state_1.json --out
put .\deltas\
=====
AUTHORIZATION STATE COMPARISON
=====
Current State: .\state_2.json
Previous State: .\state_1.json

[*] Building graphs ...
[*] Loading authorization state from: .\state_2.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-04 19:08:42
[+] Graph built successfully
    Nodes: 88
    Edges: 262
[+] Graph data exported to: deltas\state_2_graph.json
[*] Loading authorization state from: .\state_1.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-04 18:35:19
[+] Graph built successfully
    Nodes: 88
    Edges: 261
[+] Graph data exported to: deltas\state_1_graph.json
[*] Analyzing states ...
```

Terminal output `_main.py` orchestrates comparison execution - `graph_builder.py` processes both states

```
[*] Analyzing states ...  
[*] Loading graph from: deltas\state_2_graph.json  
    [+] Loaded 88 nodes  
    [+] Loaded 262 edges  
    Domain: BLUES.LOCAL  
    Collection Date: 2026-05-04 19:08:42  
    [+] Graph loaded: 88 nodes, 262 edges  
[*] Loading graph from: deltas\state_1_graph.json  
    [+] Loaded 88 nodes  
    [+] Loaded 261 edges  
    Domain: BLUES.LOCAL  
    Collection Date: 2026-05-04 18:35:19  
    [+] Graph loaded: 88 nodes, 261 edges  
[*] Comparing states ...
```

Terminal output a .main.py –comparison analyzing the change in states

```
{
  "current_state": {
    "da_paths": 5,
    "total_edges": 263,
    "kerberoastable": 3
  },
  "previous_state": {
    "da_paths": 4,
    "total_edges": 262,
    "kerberoastable": 3
  },
  "changes": {
    "new_da_paths": [
      "User:rsharma"
    ],
    "removed_da_paths": [],
    "new_edges": 1,
    "removed_edges": 0,
    "new_kerberoastable": []
  },
  "delta": {
    "da_paths": 1,
    "edges": 1,
    "kerberoastable": 0
  }
}
```

Data stored in state_comparison.json

Security evolution

Though, mdsiraj at first glance appears to be a standard user by membership, the granted Extended right makes it a node in a possible attack path to DA. This is because ResetPassword does not require the knowledge of previous password.

Therefore, if an attacker has compromised the mdsiraj account can possibly change credentials of vkohli and compromise DA.

mdsiraj [has ResetPassword] → vkohli → [has DA priv due to group nesting] → Domain Admins

State 3 vs State 2

Authorization State 3 follows from State 2 with user rsharma being granted WriteDacl on mdsiraj.

rsharma :

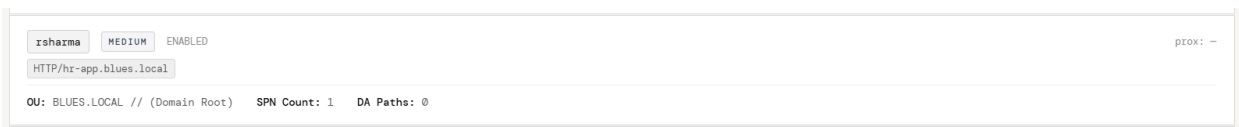
member_of HR group

GenericAll on HR group

ReadProperty on Administrator group

WriteDacl on mdsiraj (authorization state 3)

State 2 report reveals also rsharma is a Kerberoastable identity.



Comparing the states

PowerShell command

```
python main.py --compare state_3.json state_2.json --output .\deltas\
```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py --compare .\state_3.json .\state_2.json --out
put .\deltas\
=====
AUTHORIZATION STATE COMPARISON
=====
Current State: .\state_3.json
Previous State: .\state_2.json

[*] Building graphs ...
[*] Loading authorization state from: .\state_3.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-05 15:10:49
[+] Graph built successfully
    Nodes: 88
    Edges: 263
[+] Graph data exported to: deltas\state_3_graph.json
[*] Loading authorization state from: .\state_2.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-04 19:08:42
[+] Graph built successfully
    Nodes: 88
    Edges: 262
[+] Graph data exported to: deltas\state_2_graph.json
[*] Analyzing states ...
[*] Loading graph from: deltas\state_3_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 263 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-05 15:10:49
    [+] Graph loaded: 88 nodes, 263 edges

```

Terminal output `main.py` orchestrates comparison execution - `graph_builder.py` processes both states

```

    Collection Date: 2026-05-04 19:08:42
[+] Graph loaded: 88 nodes, 262 edges
[*] Comparing states ...

=====
COMPARISON RESULTS
=====

Current State (.\state_3.json):
    Domain Admin Paths:    5
    Total Edges:           263
    Kerberoastable:        3

Previous State (.\state_2.json):
    Domain Admin Paths:    4
    Total Edges:           262
    Kerberoastable:        3

Deltas:
    DA Paths:              +1
    Edges:                  +1
    Kerberoastable:        +0

    New DA Paths (1):
        + User:rsharma

[+] Comparison saved: deltas\state_comparison.json

```

Terminal output a `main.py --comparison` results

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	5/4/2026	6:53 PM	111285	state_0_graph.json
-a----	5/4/2026	7:20 PM	113291	state_1_graph.json
-a----	5/5/2026	3:37 PM	113581	state_2_graph.json
-a----	5/5/2026	3:37 PM	113864	state_3_graph.json
-a----	5/5/2026	3:37 PM	465	state_comparison.json

ouput files

```

state_comparison.json - Notepad
File Edit Format View Help
{
  "current_state": {
    "da_paths": 5,
    "total_edges": 263,
    "kerberoastable": 3
  },
  "previous_state": {
    "da_paths": 4,
    "total_edges": 262,
    "kerberoastable": 3
  },
  "changes": {
    "new_da_paths": [
      "User:rsharma"
    ],
    "removed_da_paths": [],
    "new_edges": 1,
    "removed_edges": 0,
    "new_kerberoastable": []
  },
  "delta": {
    "da_paths": 1,
    "edges": 1,
    "kerberoastable": 0
  }
}

```

Data stored in state_comparison.json

Security evolution

WriteDacl is almost a full control permission. This is because it allows the user to add, change or modify the ACE of the target object (discussed in chapter State 1). In this scenario rsharma being a Kerberoastable identity amplifies the risk. If an attacker can compromise rsharma with a Kerberoasting attack, the remainder of the privilege escalation chain becomes significantly vulnerable.

rsharma -[WriteDacl] → mdsraj -[has ResetPassword] → vkohli -[has DA priv due to group nesting] → Domain Admins

Possible attack scenario:

- *rsharma compromised by Kerberoasting attack*
- *attacker uses WriteDacl permission on mdsraj to compromise in possibly multiple ways*
 - *grant self GenericAll to gain full control*
 - *ResetPassword*
 - *Abusing certificate templates: using ESC4 to make it vulnerable to ESC1*
 - *Persistence: modify the ACLs for future access*
 - *execute malicious code on systems which mdsraj has access to*
- *compromised mdsraj can easily provide a launchpad to compromise vkohli and eventually Domain Admins*

State 4 vs State 3

ResetPassword extended right is granted to user rjadeja on rsharma.

rjadeja before this grant was a standard or can even be considered lowpriv user a membership and WriteProperty on IT group.

Comparing the states

PowerShell command

```
python main.py --compare state_4.json state_3.json --output .\deltas\
```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py --compare .\state_4.json .\state_3.json --out
put .\deltas\
=====
AUTHORIZATION STATE COMPARISON
=====
Current State: .\state_4.json
Previous State: .\state_3.json

[*] Building graphs ...
[*] Loading authorization state from: .\state_4.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57
[+] Graph built successfully
    Nodes: 88
    Edges: 264
[+] Graph data exported to: deltas\state_4_graph.json
[*] Loading authorization state from: .\state_3.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 17:52:19
[+] Graph built successfully
    Nodes: 88
    Edges: 263
[+] Graph data exported to: deltas\state_3_graph.json
[*] Analyzing states ...
[*] Loading graph from: deltas\state_4_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 264 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57

```

Terminal output `main.py` orchestrates comparison execution - `graph_builder.py` processes both states

```

    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 17:52:19
[+] Graph loaded: 88 nodes, 263 edges
[*] Comparing states ...

=====
COMPARISON RESULTS
=====

Current State (.\state_4.json):
    Domain Admin Paths: 6
    Total Edges: 264
    Kerberoastable: 3

Previous State (.\state_3.json):
    Domain Admin Paths: 5
    Total Edges: 263
    Kerberoastable: 3

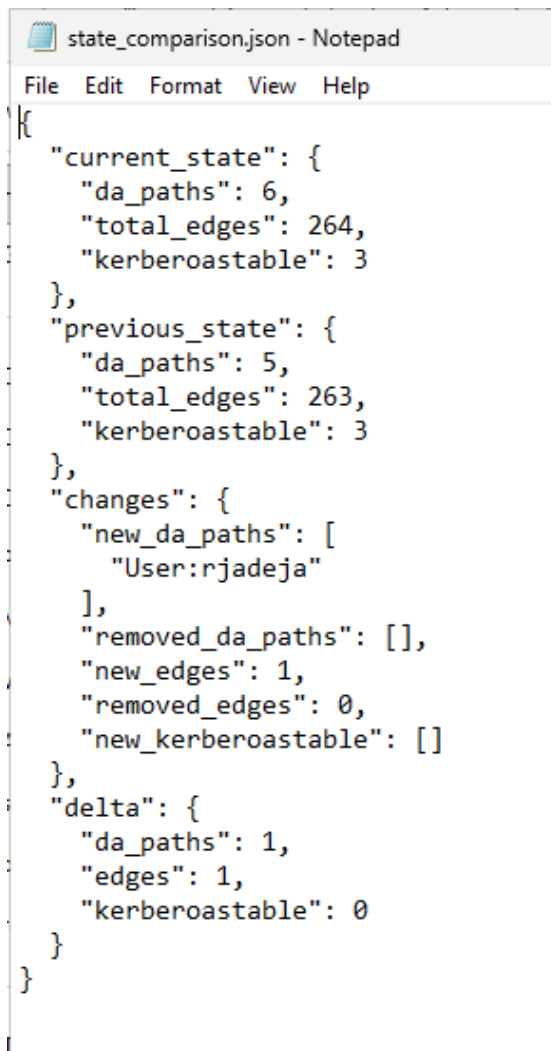
Deltas:
    DA Paths: +1
    Edges: +1
    Kerberoastable: +0

New DA Paths (1):
    + User:rjadeja

[+] Comparison saved: deltas\state_comparison.json

```

Terminal output a `main.py --comparison` results



```
{
  "current_state": {
    "da_paths": 6,
    "total_edges": 264,
    "kerberoastable": 3
  },
  "previous_state": {
    "da_paths": 5,
    "total_edges": 263,
    "kerberoastable": 3
  },
  "changes": {
    "new_da_paths": [
      "User:rjadeja"
    ],
    "removed_da_paths": [],
    "new_edges": 1,
    "removed_edges": 0,
    "new_kerberoastable": []
  },
  "delta": {
    "da_paths": 1,
    "edges": 1,
    "kerberoastable": 0
  }
}
```

Data stored in state_comparison.json

Security evolution:

As of State 3 we can see an escalation chain formed *rsharma* → *mdsraj* → *vkohl* → *Domain Admins*.

State comparison clearly detects an additional DA attack path. It also registers the edge created between *rjadeja* and *rsharma*.

The chain gets longer as *rjadeja* is granted the ResetPassword on *rsharma*.

As we have discussed, ResetPassword allows the user to change the password of the target object without knowledge of the earlier password. Therefore, a compromised *rjadeja* can easily facilitate compromise of *rsharma* and the entire chain:

~~mda~~ [ResetPassword] → ~~rshama~~ [Me Dad] → ~~mdsra~~
 [ResetPassword] → vko h [Group neshg] → Domain Admins

State 5 vs State 4

Authorization states 2,3,4 saw user on user ACL permissions applied. But in State 5 user rashwin was granted WriteProperty on the custom group Director.

Comparing the states

Powershell command

```
python main.py --compare state_5.json state_4.json .\deltas\
```

```

PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py --compare .\state_5.json .\state_4.json --out
put .\deltas\

=====
AUTHORIZATION STATE COMPARISON
=====
Current State: .\state_5.json
Previous State: .\state_4.json

[*] Building graphs ...
[*] Loading authorization state from: .\state_5.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
[+] Graph built successfully
    Nodes: 88
    Edges: 265
[+] Graph data exported to: deltas\state_5_graph.json
[*] Loading authorization state from: .\state_4.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57
[+] Graph built successfully
    Nodes: 88
    Edges: 264
[+] Graph data exported to: deltas\state_4_graph.json
[*] Analyzing states ...
[*] Loading graph from: deltas\state_5_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 265 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
[+] Graph loaded: 88 nodes, 265 edges
  
```

Terminal output main.py orchestrates comparison execution - graph_builder.py processes both states

```

    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:02:57
[+] Graph loaded: 88 nodes, 264 edges
[*] Comparing states ...

=====
COMPARISON RESULTS
=====

Current State (.\state_5.json):
    Domain Admin Paths: 7
    Total Edges: 265
    Kerberoastable: 3

Previous State (.\state_4.json):
    Domain Admin Paths: 6
    Total Edges: 264
    Kerberoastable: 3

Deltas:
    DA Paths: +1
    Edges: +1
    Kerberoastable: +0

    New DA Paths (1):
    + User:rashwin

[+] Comparison saved: deltas\state_comparison.json

```

Terminal output a_m ain.py --comparison results

File Edit Format View Help

```
"current_state": {
  "da_paths": 7,
  "total_edges": 265,
  "kerberoastable": 3
},
"previous_state": {
  "da_paths": 6,
  "total_edges": 264,
  "kerberoastable": 3
},
"changes": {
  "new_da_paths": [
    "User:rashwin"
  ],
  "removed_da_paths": [],
  "new_edges": 1,
  "removed_edges": 0,
  "new_kerberoastable": []
},
"delta": {
  "da_paths": 1,
  "edges": 1,
  "kerberoastable": 0
}
```

Data stored in state_comparison.json

Security evolution:

As we had seen in State 1 that the Director group was made a member of Domain Admins group, and as a result the members of Director had inherited the DA rights due to this group nesting.

rashwin who previously had:

membership of IT and Finance

GenericWrite on IT

WriteProperty on Finance

Now, granted a WriteProperty on Director creates a possible Domain Admin escalation path.

This is because WriteProperty on a group can be extremely dangerous depending on which attribute(s) the ACE applies to.

If the ACE allows writing the member attribute:

```
WriteProperty -> member
```

then the user can:

- add themselves to the group
- add arbitrary users
- remove users

This is one of the most common AD privilege-escalation paths.

Possible escalation method:

rashwin-[WriteProperty(member)] → Director

- *add self to Director*
- *inherit Domain Admin privilege (as Director is nested in Domain Admins group)*

State 5 vs state 0

State 0 was the baseline, where the custom users were only granted memberships of the custom groups and no ACLs had been applied on them.

Therefore, there was no realistic identity based escalation path to Domain Admins.

From State 1 onwards ACLs are applied, group nesting and critical privileges are granted to standard or low privilege users, to see how AD-AuthGraph performs in detecting these changing authorization states and analyse them to determine if new possible escalation paths to Domain Admins are created.

Comparing the states

Powershell command

```
python main.py --compare state_5.json state_0.json .\deltas\
```

```
PS C:\Users\Administrator\AD_Authorization_Graph_project> python .\main.py --compare .\state_5.json .\state_0.json --out
put .\deltas\
=====
AUTHORIZATION STATE COMPARISON
=====
Current State: .\state_5.json
Previous State: .\state_0.json

[*] Building graphs ...
[*] Loading authorization state from: .\state_5.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
[+] Graph built successfully
    Nodes: 88
    Edges: 265
[+] Graph data exported to: deltas\state_5_graph.json
[*] Loading authorization state from: .\state_0.json
    Domain: BLUES.LOCAL
    Collection Date: 2026-04-13 17:49:05
[+] Graph built successfully
    Nodes: 87
    Edges: 230
[+] Graph data exported to: deltas\state_0_graph.json
[*] Analyzing states ...
[*] Loading graph from: deltas\state_5_graph.json
    [+] Loaded 88 nodes
    [+] Loaded 265 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-05-06 18:28:11
```

Terminal output `main.py` orchestrates comparison execution - `graph_builder.py` processes both states

```
[+] Graph loaded: 88 nodes, 265 edges
[*] Loading graph from: deltas\state_0_graph.json
    [+] Loaded 87 nodes
    [+] Loaded 230 edges
    Domain: BLUES.LOCAL
    Collection Date: 2026-04-13 17:49:05
[+] Graph loaded: 87 nodes, 230 edges
[*] Comparing states ...
```

Nodes and edges calculated for both state 5 and state 0 graphs

```
=====
COMPARISON RESULTS
=====
```

```
Current State (.\state_5.json):
```

```
Domain Admin Paths: 7
Total Edges:        265
Kerberoastable:     3
```

```
Previous State (.\state_0.json):
```

```
Domain Admin Paths: 1
Total Edges:        230
Kerberoastable:     2
```

```
Deltas:
```

```
DA Paths:           +6
Edges:              +35
Kerberoastable:     +1
```

```
New DA Paths (6):
```

```
+ User:mdsiraj
+ User:vkohli
+ User:rsharma
+ User:jbumrah
+ User:rashwin
+ User:rjadeja
```

```
New Kerberoastable (1):
```

```
+ User:rsharma
```

```
[+] Comparison saved: deltas\state_comparison.json
```

Terminal output a_m ain.py --comparison results

```
state_comparison.json - Notepad
File Edit Format View Help
{
  "current_state": {
    "da_paths": 7,
    "total_edges": 265,
    "kerberoastable": 3
  },
  "previous_state": {
    "da_paths": 1,
    "total_edges": 230,
    "kerberoastable": 2
  },
  "changes": {
    "new_da_paths": [
      "User:mdsiraj",
      "User:vkohli",
      "User:rsharma",
      "User:jbumrah",
      "User:rashwin",
      "User:rjadeja"
    ],
    "removed_da_paths": [],
    "new_edges": 38,
    "removed_edges": 3,
    "new_kerberoastable": [
      "User:rsharma"
    ]
  },
  "delta": {
    "da_paths": 6,
    "total_edges": 35,
    "kerberoastable": 1
  }
}
```

Data stored in state_comparison.json

Security evolution of authorization states:

State 0 - baseline - custom users, custom groups created, no ACLs applied.

State 1 - ACLs applied. Director group nested in Domains. vkohli and jbumrah inherit DA rights via membership of Director group.

State 2 - user mdsiraj granted ResetPassword on vkohli. DA escalation path detected: mdsiraj → vkohli → Domain Admins

State 3 - rsharma granted WriteDacl on mdsiraj. DA escalation path detected: rsharma → mdsiraj → vkohli → Domain Admins

State 4: rjadeja granted ResetPassword on rsharma. DA escalation path detected:
rjadeja → rsharma → mdsiraj → vkohli → Domain Admins

State 5: For this authorization state a user rashwin is granted WriteProperty on group Director. DA escalation path detected: rashwin → Director → Domain Admins

AD-AuthGraph accurately detects the change in authorisation for each state. A comparison between state 0 and state 5 it accurately lists the source (originating) nodes for DA escalation paths that have evolved over the changing states.

AD-AuthGraph
End of Report